

ΑΝΑΓΝΩΡΙΣΗ ΠΡΟΤΥΠΩΝ

Τελική Εργασία



Φοιτητής: Ιατρού Κωνσταντίνος 58071

Καθηγητής: Θεοδωρακόπουλος Ηλίας

ΠΕΡΙΕΧΟΜΕΝΑ

Μέρος 1 - Ταξινόμηση Εικόνων.....	Σελ. 2
Μέρος 2 - Πρόβλεψη πρόσδεσης χημικών μορίων σε βιολογικούς υποδοχείς.....	Σελ. 19

Μέρος 1 - Ταξινόμηση Εικόνων

Εκφώνηση:

Στο πρώτο μέρος καλείστε να επιλύσετε ένα πρόβλημα ταξινόμησης εικόνων που προκύπτουν από αλγορίθμους ανίχνευσης προσώπων, με στόχο την επιβεβαίωση χρήσης μάσκας. Τα δεδομένα που θα χρησιμοποιήσετε βρίσκονται στο αρχείο Mask_DB.zip, και αποτελούνται από 1044 εικόνες προσώπων χωρίς τη χρήση μάσκας (without_mask), ισάριθμες εικόνες προσώπων που χρησιμοποιούν μάσκα (with_mask), και 56 εικόνες προσώπων που χρησιμοποιούν μάσκα εσφαλμένα (mask_incorrect_use).

a. Ως δεδομένα εκπαίδευσης, επικύρωσης και δοκιμής θα χρησιμοποιηθούν μόνο εικόνες από τις κλάσεις with_mask και without_mask με τις αντίστοιχες ετικέτες. Χωρίστε τα δεδομένα τυχαία σε 60% δεδομένα εκπαίδευσης, 20% δεδομένα επικύρωσης και 20% δεδομένα δοκιμής.

b. Υλοποιείστε και εκπαιδεύστε ένα σύστημα αναγνώρισης προτύπων χρησιμοποιώντας κατάλληλους αλγορίθμους της επιλογής σας συμβατικούς ή/και νευρωνικούς. Δικαιολογήστε το σκεπτικό σας.

c. Καθορίστε τις βέλτιστες παραμέτρους του μοντέλου σας (πχ. Αριθμό ελάττωσης διαστάσεων, παραμέτρους ταξινομητών, μοντέλα δικτύων κλπ.) χρησιμοποιώντας το σύνολο επικύρωσης. Δώστε μερικά παραδείγματα παραλλαγών που δοκιμάσατε και την επίδοσή τους στο validation set. Για το καλύτερο μοντέλο σας υπολογίστε το σφάλμα ταξινόμησης στο σύνολο δοκιμής, καθώς και τις καμπύλες precision-recall και το AUC.

d. Εφαρμόστε το εκπαιδευμένο μοντέλο σας στις εικόνες της κλάσης «mask_incorrect_use» και υπολογίστε το ποσοστό αυτών που ταξινομούνται εσφαλμένα ως αποδεκτή χρήση μάσκας. Προτείνετε τρόπους που μπορεί να αυξηθεί ο αριθμός των εικόνων της κλάσης mask_incorrect_use που ταξινομούνται ως μη χρήση μάσκας, χωρίς να επηρεαστούν σημαντικά τα αποτελέσματα του test set, χρησιμοποιώντας μόνο τα υπάρχοντα δεδομένα. Δοκιμάστε κάτι από αυτά που προτείνετε και δείξτε εάν έχει το επιθυμητό αποτέλεσμα.

Επίλυση:

Ερώτημα a

Σε πρώτη φάση έπρεπε μόνο να γίνει η εισαγωγή των δεδομένων. Ταυτόχρονα με την εισαγωγή ήταν απαραίτητη και η δημιουργία ετικέτας για κάθε δεδομένων των δύο φακέλων with_mask και without_mask. Μετά από πολλές προσπάθειες βρέθηκε ένας, μη βέλτιστος τρόπος (κατά την άποψη) για την εισαγωγή των δεδομένων.

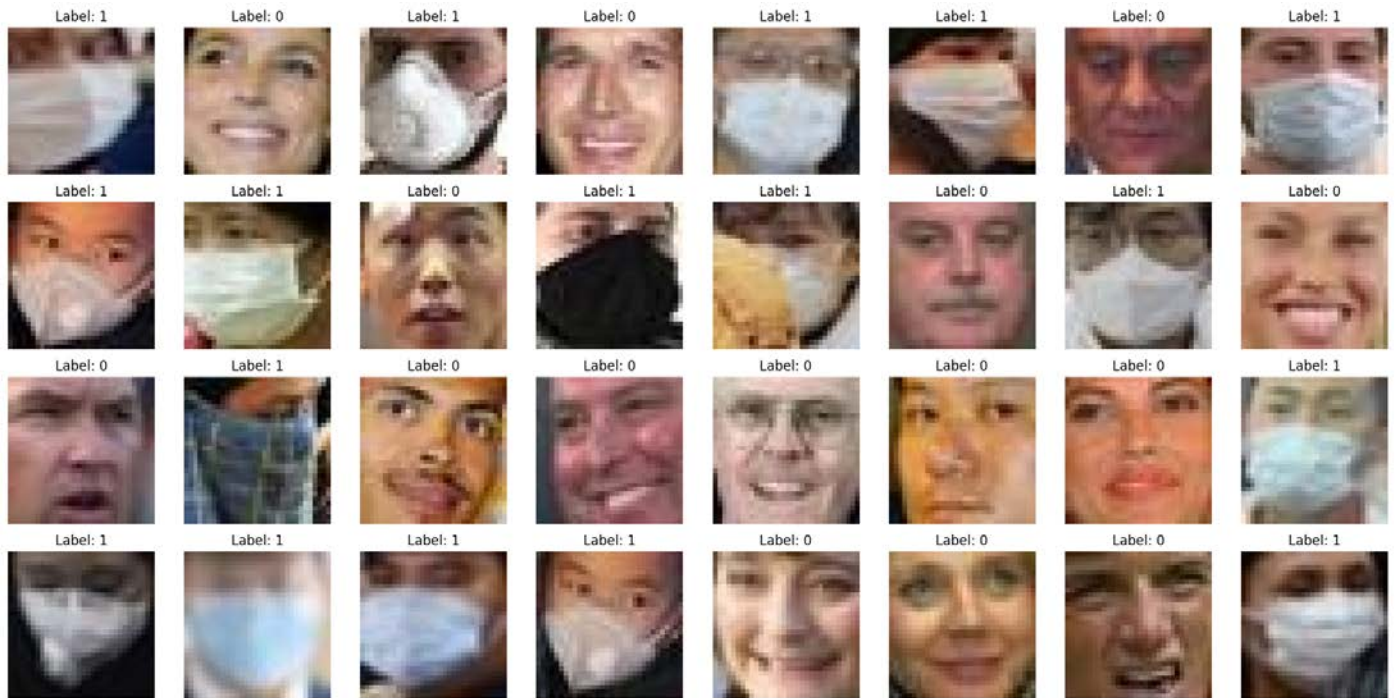
Αρχικά δημιουργήθηκε μία λίστα με τα paths για κάθε μία από τις εικόνες των δύο φακέλων ενώ ταυτόχρονα όσο εισάγαμε paths στην λίστα, τοποθετούσαμε σε μία άλλη λίστα με την ίδια σειρά τις τιμές των ετικετών. Έτσι καταλήξαμε να έχουμε την κλάση without_mask με ετικέτα 0 και την κλάση with_mask με ετικέτα 1.

Στην συνέχεια αυτές οι λίστες χωρίστηκαν όπως ζητήθηκε σε 3 μικρότερες (η κάθε μία), δημιουργώντας μία λίστα με τα paths για τις εικόνες του σετ εκπαίδευσης και την αντίστοιχη λίστα με τις ετικέτες τους, οι οποίες έχουν το 60% των στοιχείων της αρχικής λίστας, και οι λίστες με τα paths των δεδομένων για τα σετ επαλήθευσης και δοκιμής με τις αντίστοιχες λίστες με των ετικετών τους με το 20% των στοιχείων της αρχικής λίστας η κάθε μία.

Στην συνέχεια χρησιμοποιήθηκαν τα paths και μία κλάση που υλοποιήθηκε στα πλαίσια της εργασίας (MaskDataset) η οποία παίρνει τα paths και με την βοήθεια της βιβλιοθήκης PIL και συγκεκριμένα του module Image ανοίγει τις εικόνες και τις εισάγει ως δεδομένα στα αντικείμενα τύπου MaskDataset μαζί με την αντίστοιχη ετικέτα της εικόνας.

Τέλος και τα τρία Dataset (train, validation, test) φορτώνονται σε αντικείμενα τύπου DataLoader, τα οποία μπορούν να χρησιμοποιηθούν από την βιβλιοθήκη PyTorch για την εκπαίδευση νευρωνικών δικτύων.

Τυπώθηκαν και μερικές εικόνες για να σιγουρευτούμε ότι δεν έχει προκύψει κάποιο πρόβλημα:



Ερώτημα b

Για την υλοποίηση συστήματος ταξινόμησης επέλεξα την υλοποίηση και την εκπαίδευση κάποιου νευρωνικού δικτύου. Επέλεξα το νευρωνικό δίκτυο για τους παρακάτω λόγους:

1. Τα νευρωνικά δίκτυα έχουν υψηλή απόδοση στην αναγνώριση πολύπλοκων δεδομένων. Ειδικότερα τα συνελκτικά νευρωνικά δίκτυα (CNNs) έχουν ιδιαίτερα καλή απόδοση στον εντοπισμό και την αναγνώριση πολύπλοκων μοτίβων πάνω στα δεδομένα-εικόνες, όπως στο συγκεκριμένο σετ δεδομένων τα πρόσωπα και το αν φοράνε μάσκα ή όχι.
2. Τα συνελκτικά νευρωνικά δίκτυα έχουν ιδιαίτερα μεγάλη ικανότητα να αναγνωρίζουν αυτόματα ποια χαρακτηριστικά είναι σημαντικότερα στις εικόνες που δίνονται ως δεδομένα. Έτσι δεν υπάρχει πλέον ανάγκη χειροκίνητου εντοπισμού και εξαγωγής των συγκεκριμένων χαρακτηριστικών, κάτι που είναι συνήθως απαραίτητο στους συμβατικούς αλγορίθμους.
3. Δίνουν την δυνατότητα να μην δημιουργήσουμε ένα σύστημα από το 0, αλλά να χρησιμοποιήσουμε κάποιο ήδη προ-εκπαιδευμένο μοντέλο συνελκτικού νευρωνικού δικτύου, το οποίο θα έχει ήδη εκπαιδευτεί σε ένα πολύ μεγάλο σύνολο δεδομένων (συνήθως είναι το ImageNet). Έτσι το ήδη εκπαιδευμένο μοντέλο θα χρειαστεί ελάχιστη περαιτέρω εκπαίδευση στα δεδομένα του δικού μας προβλήματος.
4. Ακόμα, τα CNNs είναι γνωστά για την ικανότητα τους για καλή γενίκευση σε νέα δεδομένα του ίδιου προβλήματος, παρέχοντας υψηλή ακρίβεια και σχετικά σταθερή απόδοση σε διαφορετικές και πραγματικές συνθήκες.
5. Υπάρχουν αρκετά εργαλεία μηχανικής μάθησης τα οποία έχουν μεγάλη κοινότητα που τα υποστηρίζουν πράγμα που διευκολύνει την επίλυση προβλημάτων που μπορεί να προκύψουν κατά την διάρκεια της υλοποίησης.
6. Προσφέρουν ευελιξία στην επιλογή αρχιτεκτονικής, παραμέτρων για την βελτίωση της απόδοσης του ώστε να ταιριάζει στις απαιτήσεις το κάθε πρόβλημα που μπορεί να εφαρμοστεί.
7. Τέλος, η επιλογή γίνεται εν μέρει και για προσωπική προτίμηση προς τα νευρωνικά δίκτυα ως διαδικασία υλοποίησης και εφαρμογής σε προβλήματα.

Αφού αναλύθηκαν οι λόγοι που επιλέχθηκε ως σύστημα ταξινόμησης ένα μοντέλο νευρωνικού δικτύου, μπορούμε να προχωρήσουμε στην πρώτη υλοποίηση της αρχιτεκτονικής του νευρωνικού δικτύου που έγινε στα πλαίσια του ερωτήματος αυτού:

```

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 8, kernel_size=3, stride=1, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2, padding=0)
        self.fc1 = nn.Linear(8 * 16 * 16, 16)
        self.fc2 = nn.Linear(16, 1) # 2 Κλάσεις: with_mask, without_mask (Δυναμική ταξινόμηση)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = x.view(-1, 8 * 16 * 16) # Flatten tensor
        x = torch.relu(self.fc1(x))
        x = torch.sigmoid(self.fc2(x))
        return x

```

Επειδή πρόκειται για την πρώτη απόπειρα δημιουργίας ενός νευρωνικού για την επίλυση του προβλήματος αναγνώρισης την χρήση ή μη χρήση μάσκας, η υλοποίηση είναι αρκετά απλή και αποτελείται από:

- 1 Συνελικτικό επίπεδο (**self.conv1 = nn.Conv2d(3, 8, kernel_size=3, stride=1, padding=1)**). Συγκεκριμένα αυτό το επίπεδο εκτελεί συνελίζεις στην εικόνα εισόδου για να εξάγει χαρακτηριστικά. Η τιμή 3 ορίζει τον αριθμό των εισόδων (input layer) και αναφέρεται στις τιμές των τριών καναλιών που έχει κάθε pixel της εικόνας καθώς οι εικόνες που μας δόθηκαν στο σύνολο δεδομένων έχουν μορφή RGB. Η τιμή 8 αναφέρεται στους νευρώνες του συνελικτικού επιπέδου, καθένας εκ των οποίων εκπαιδεύεται για την εφαρμογή ενός φίλτρου.
- 1 Επίπεδο μείωσης διαστάσεων (**self.pool = nn.MaxPool2d(kernel_size=2, stride=2, padding=0)**). Το επίπεδο αυτό εφαρμόζει την τεχνική Max Pooling, δηλαδή επιλέγει την μέγιστη τιμή από κάθε τμήμα του χάρτη των χαρακτηριστικών που καλύπτεται από τον Kernel του επιπέδου. Αυτό στην ουσία μειώνει της διαστάσεις, διατηρώντας τις σημαντικότερες τιμές, επιλέγοντας στην ουσία τα σημαντικότερα χαρακτηριστικά. Έτσι μειώνεται το υπολογιστικό φόρτο και τον χώρο που θα απαιτηθεί από το επόμενο επίπεδο. Ακόμη το συγκεκριμένο βήμα βοηθάει στην αποφυγή της υπερπροσαρμογής του μοντέλου στα δεδομένα εκπαίδευσης.
- 2 Fully connected επίπεδα, ένα κρυφό (**self.fc1 = nn.Linear(8 * 16 * 16, 16)**) και το επίπεδο εξόδου (**self.fc2 = nn.Linear(16, 1)**). Το πρώτο παίρνει τα δεδομένα που δίνει το συνελικτικό επίπεδο τα οποία μετατρέπονται σε διάνυσμα και το δεύτερο εκτελεί την ταξινόμηση με δυαδικό τρόπο.
- Συναρτήσεις ενεργοποίησης. Για το συνελικτικό επίπεδο και για το πρώτο fully connected επίπεδο εφαρμόζεται η συνάρτηση ενεργοποίησης ReLU, ενώ για το output επίπεδο εφαρμόζεται η συνάρτηση ενεργοποίησης sigmoid. Η σιγμοειδής συνάρτηση στο επίπεδο εξόδου εφαρμόζεται για να κανονικοποίηση την έξοδο του τελευταίου επιπέδου στο διάστημα [0, 1]. Η συνάρτηση ReLU για τα άλλα επίπεδα επιλέχθηκε λόγω των πλεονεκτημάτων που προσφέρει, τα οποία είναι:
 1. Αποτρέπει το πρόβλημα εξαφάνισης της κλίσης, δίνοντας την δυνατότητα σε βαθιά νευρωνικά δίκτυα να μάθουν πιο γρήγορα και αποτελεσματικά σε αντίθεση με την Sigmoid της οποίας η κλίση εξαφανίζεται από ένα σημείο και μετά μη επιτρέποντας το δίκτυο να συνεχίσει την εκμάθηση το ίδιο αποδοτικά έως και καθόλου.
 2. Έχει μικρό υπολογιστικό κόστος και ειδικά σε σχέση με την σιγμοειδή η διαφορά είναι αρκετά μεγάλη σε υπολογισμούς (σε βάθος χρόνου).

Πριν προχωρήσουμε, οφείλουμε να εξηγήσουμε μερικές έννοιες που εμφανίζονται στα πρώτα 2 επίπεδα (Συνελικτικό και Μείωσης διαστάσεων). Οι έννοιες αυτές είναι:

Kernel (Kernel size): Στα ελληνικά πυρήνας, είναι ένα μικρό παράθυρο που κινείται πάνω στην εικόνα και καλύπτει ένα μέρος της, στην προκειμένη περίπτωση έχει μέγεθος 3x3 και συνεπώς καλύπτει ένα τετράγωνο με 9 pixel κάθε φορά. Αυτό το παράθυρο παίρνει ως είσοδο τα pixel της εικόνας που καλύπτει και εκτελεί την συνέλιξη. Το kernel size συνεπώς αναφέρεται στον αριθμό των pixel που απαρτίζουν την κάθε πλευρά του τετράγωνου kernel. Στην περίπτωση

του επιπέδου Pool το kernel size καθορίζει το μέγεθος της περιοχής που θα εφαρμοστεί το pooling και έχει άμεση σχέση με το ποσοστό κατά το οποίο θα μειωθεί το μέγεθος της εικόνας για να περάσει στο επόμενο επίπεδο του δικτύου.

Stride: Πρακτικά είναι το βήμα κατά το οποίο θα μετακινηθεί ο kernel κατά το σκανάρισμα της εικόνας. Ορίζοντας την τιμή ένα, στην ουσία ορίζουμε στον πυρήνα να μετακινείται 1 βήμα την φορά. Αν το βήμα μεγαλώσει θα συμβούν 2 πράγματα, θα εκτελεστεί η κίνηση του πυρήνα γρηγορότερα, αλλά θα μειωθούν και οι διαστάσεις του χάρτη χαρακτηριστικών που θα παράγει το φίλτρο.

Padding: Το συγκεκριμένο χαρακτηριστικό αφορά την προσθήκη γραμμών γύρω από την εικόνα. Το συγκεκριμένο χαρακτηριστικό είναι απαραίτητο για την διατήρηση του μεγέθους της εικόνας. Αν δεν εφαρμοστεί μετά από κάθε συνελικτικό επίπεδο οι διαστάσεις της εικόνας θα μειωνόντουσαν, ανάλογα με το μέγεθος του πυρήνα, μην επιτρέποντας την χρήση δικτύων πολύ μεγάλου βάθους.

Στην συνέχεια δημιουργώ ένα μοντέλο βασισμένο στην αρχιτεκτονική του νευρωνικού δικτύου SimpleCNN και ορίζω τις παραμέτρους που είναι απαραίτητοι για την εκπαίδευσή του:

Ρυθμός μάθησης (Learning Rate): Η συγκεκριμένη παράμετρος καθορίζει το μέγεθος των βημάτων που θα γίνεται για την ενημέρωση των βαρών του δικτύου κατά την διάρκεια της εκπαίδευσής του. Η επιλογή πρέπει να γίνεται με προσοχή, ώστε να μην επιλεγεί πολύ μεγάλη τιμή ρυθμού μάθησης, μην επιτρέποντας το μοντέλο να φτάσει στην βέλτιστη δυνατή κατάσταση, αλλά να μην επιλεγεί ούτε πολύ μικρή τιμή αναγκάζοντας το μοντέλο να εκτελέσει πολύ μεγάλο αριθμό επαναλήψεων για να φτάσει στην επιθυμητή βέλτιστη κατάσταση.

Συνάρτηση απώλειας (Loss Function): Η συνάρτηση που επιλέχθηκε για το συγκεκριμένο δίκτυο και θα προτιμάται γενικότερα κατά την διάρκεια της άσκησης είναι η Binary Cross Entropy Loss, η οποία είναι κατάλληλη για ταξινομήσεις που γίνονται με δυαδικό τρόπο, ακριβώς όπως συμβαίνει στο εν λόγω πρόβλημα. Πρακτικά μετρά την διαφορά μεταξύ των τιμών που προβλέπει το μοντέλο και των πραγματικών ετικετών, όντας περισσότερο τιμωρητική απέναντι σε λανθασμένες προβλέψεις του μοντέλου οι οποίες γίνονται με «αυτοπεποίθηση». Ο μαθηματικός τύπος που ορίζει την συγκεκριμένη συνάρτηση είναι ο παρακάτω:

$$BCELoss = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log \hat{y}_i + (1 - y_i) \cdot \log(1 - \hat{y}_i)$$

όπου N ο αριθμός των δεδομένων της παρτίδας (batch), y_i η πραγματική ετικέτα του δεδομένου i και $\hat{y}_i$ η πρόβλεψη του μοντέλου.

Αποτελεί μία σταθερή συνάρτηση απώλειας και αποφεύγει την εφαρμογή ακραίων αλλαγών στα βάρη κατά την διάρκεια της εκπαίδευσης, εξομαλύνοντας την διαδικασία βελτιστοποίησης του μοντέλου.

Αλγόριθμος Βελτιστοποίησης (Optimizer): Ο αλγόριθμος βελτιστοποίησης ορίζει τον τρόπο με τον οποίο θα προσπαθεί το μοντέλο, με βάσει τα αποτελέσματα μιας εποχής εκπαίδευσης ή ενός batch, να ανανεώσει τα βάρη του ώστε να κατευθυνθεί στην βέλτιστη κατάσταση κατά την οποία η ακρίβεια να είναι η μέγιστη δυνατή και η απώλεια που θα υπολογίζεται από την συνάρτηση απώλειας θα είναι η ελάχιστη δυνατή. Επιλέχθηκε ο Adam σαν αλγόριθμος βελτιστοποίησης καθώς ρυθμίζει τον ρυθμό μάθησης δυναμικά και είναι γνωστός για την καλή του απόδοση σε πολλές εφαρμογές. Πιο αναλυτικά τα πλεονεκτήματα που προσφέρει είναι:

1. Momentum: Ο Adam υπολογίζει τις μέσες τιμές των παραγώγων και των τετραγώνων των παραγώγων αυτών. Αυτό του δίνει την δυνατότητα να διατηρεί «μνήμη» για τις προηγούμενες παραγώγους, εξομαλύνοντας και σταθεροποιώντας την πορεία της βελτιστοποίησης.
2. Adaptive Learning Rate: Ο Adam προσαρμόζει το βήμα της βελτιστοποίησης για κάθε παράμετρο ανεξάρτητα, βάσει των παραγώγων τους. Αυτό δίνει την δυνατότητα για μεγάλα βήματα για τις παραμέτρους που έχουν μεγάλες τιμές στις παραγώγους τους και μικρότερα βήματα για παραμέτρους που έχουν παραγώγους με μικρές τιμές. Αυτό τον κάνει πιο αποδοτικό σε σχέση με άλλες μεθόδους, όπως για παράδειγμα είναι ο Stochastic Gradient Descent, και πιο σταθερό κατά την διαδικασία της βελτιστοποίησης.

3. Bias Correction: Εφαρμόζει μία διόρθωση σε προκατάληψη που μπορεί να υπάρχει, αντισταθμίζοντας την τάση που μπορεί να υπάρχει προς το 0.

Έχοντας ορίσει όλες τις παραμέτρους προχωράμε στην εκπαίδευση του μοντέλου για 10 εποχές:

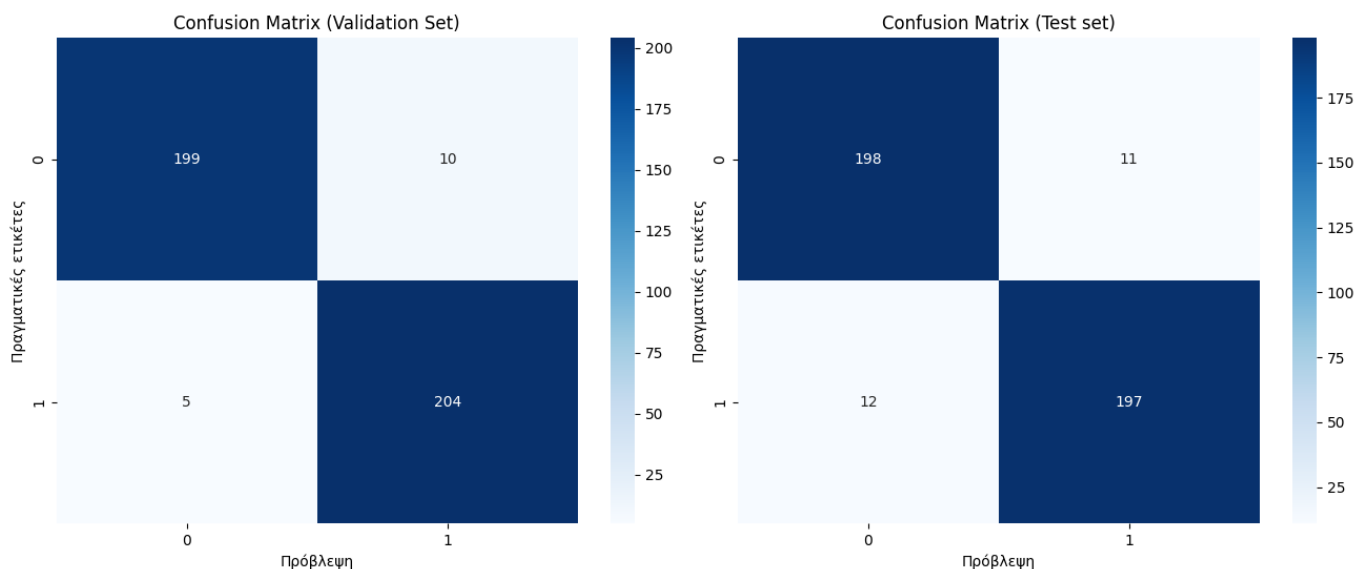
```
Epoch [1/10], Loss: 13.0084, Val Loss: 4.0845, Val Accuracy: 86.60%
Epoch [2/10], Loss: 10.1792, Val Loss: 2.8147, Val Accuracy: 91.63%
Epoch [3/10], Loss: 6.9312, Val Loss: 1.8880, Val Accuracy: 92.34%
Epoch [4/10], Loss: 4.8065, Val Loss: 1.4399, Val Accuracy: 94.26%
Epoch [5/10], Loss: 3.7270, Val Loss: 1.1130, Val Accuracy: 95.93%
Epoch [6/10], Loss: 3.2253, Val Loss: 0.9943, Val Accuracy: 96.17%
Epoch [7/10], Loss: 2.9075, Val Loss: 0.9597, Val Accuracy: 94.98%
Epoch [8/10], Loss: 2.9925, Val Loss: 1.0255, Val Accuracy: 93.78%
Epoch [9/10], Loss: 2.6169, Val Loss: 0.7651, Val Accuracy: 96.17%
Epoch [10/10], Loss: 2.4788, Val Loss: 0.6995, Val Accuracy: 96.41%
```

Η ακρίβεια που επιτυγχάνει το μοντέλο στο σετ επαλήθευσης μετά την τελευταία δεν είναι μικρή, θα μπορούσε ίσως να είναι και μεγαλύτερη. Αυτό θα προσπαθήσουμε να το βελτιώσουμε στην συνέχεια της άσκησης. Ακόμα υπολογίζεται και η ακρίβεια του μοντέλου πάνω στα δεδομένα του σετ δοκιμής:

```
Test Loss: 1.05911, Test Accuracy: 94.4976%
```

Για την ακρίβεια εδώ ισχύουν αυτά που ειπώθηκαν και για την ακρίβεια στα δεδομένα του σετ επαλήθευσης.

Ταυτόχρονα παράγονται και οι πίνακες σύγχυσης τόσο για το σετ επαλήθευσης όσο και για το σετ δοκιμής:



Εδώ παρατηρείται ότι γενικά γίνονται λίγα λάθη από το μοντέλο, αλλά στην περίπτωση του σετ επαλήθευσης τα λάθη του τύπου «Περιπτώσεις χωρίς μάσκα που ταξινομούνται ως Με μάσκα» είναι περισσότερα. Πρόκειται για σοβαρό πρόβλημα γιατί το να μην φοράει μάσκα κάποιος σε χώρους όπως τα νοσοκομεία ή σε μέρη που υπάρχουν γενικότερα άτομα με προβλήματα υγείας είναι επικίνδυνο. Όσον αφορά τις ταξινομήσεις των δεδομένων του σετ δοκιμής τα σφάλματα φαίνονται μοιρασμένα και για τις δύο κατηγορίες.

Ακόμη στα πλαίσια της αξιολόγησης κάθε μοντέλου υπολογίζεται και η καμπύλη Precision-Recall καθώς και το εμβαδόν (AUC) που ορίζεται κάτω από αυτή.

Οι καμπύλες Precision-Recall είναι ένα πολύ σημαντικό εργαλείο που χρησιμοποιείται για την αξιολόγηση των μοντέλων και των αλγορίθμων της μηχανικής μάθησης σε εφαρμογές ταξινόμησης. Για να πορχωρήσουμε σε επεξήγηση των καμπυλών, χρειάζεται πρώτα να εξηγηθούν οι όροι Precision (Ακρίβεια) και Recall (Ανάκληση).

Precision (Ακρίβεια)

Με τον όρο ακρίβεια αναφερόμαστε στο ποσοστό των σωστών θετικών προβλέψεων του μοντέλου από το σύνολο όλων των θετικών προβλέψεων του μοντέλου. Ο υπολογισμός της γίνεται με τον παρακάτω τύπο:

$$Precision = \frac{(True\ Positives)}{(True\ Positives) + (False\ Positives)}$$

Recall (Ανάκληση)

Με τον όρο ανάκληση αναφερόμαστε στο ποσοστό των θετικών προβλέψεων του μοντέλου από το σύνολο όλων των πραγματικών θετικών περιπτώσεων. Ο υπολογισμός γίνεται με τον παρακάτω τύπο:

$$Recall = \frac{(True\ Positives)}{(True\ Positives) + (False\ Negative)}$$

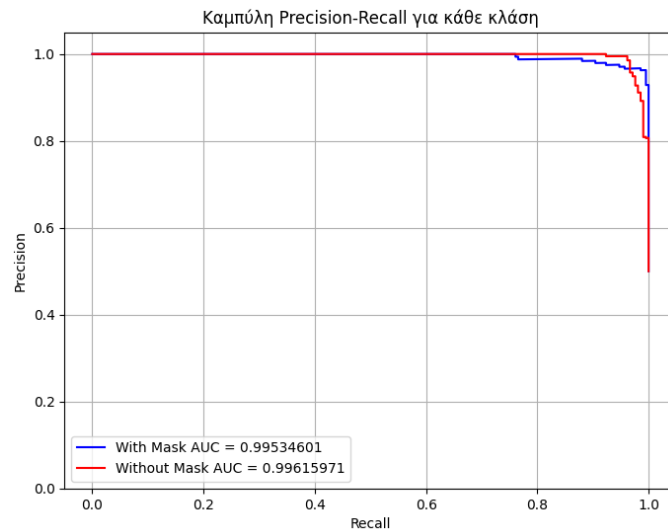
Όσον αφορά τις καμπύλες Precision-Recall, είναι ιδιαίτερα χρήσιμες σε προβλήματα ταξινόμησης (συνήθως) όπου υπάρχει ανισορροπία μεταξύ των θετικών και των αρνητικών κλάσεων και σε προβλήματα που το κόστος μεταξύ των ψευδών θετικών και των ψευδών αρνητικών δεν είναι ίσο. Στη πρώτη κατηγορία προβλημάτων χρησιμοποιούμε τις συγκεκριμένες καμπύλες όταν τα θετικά είναι λιγότερα σε αριθμό και στην δεύτερη κατηγορία όταν το κόστος των ψευδών θετικών είναι υψηλότερο.

Η καμπύλη Precision-Recall ορίζει μία ακόμα μετρική που ονομάζεται AUC-PR και είναι ίση με το εμβαδόν που ορίζεται κάτω από την καμπύλη αυτή. Η σημασία του εμβαδού είναι ότι μας αξιολογεί την σημαντική αποτελεσματικότητα του ταξινομητή σε όλα τα πιθανά κατώφλια του ταξινομητή για την ταξινόμηση ενός δεδομένου. Ακόμη μας δείχνει πόσο καλή ακρίβεια και ανάκληση διατηρεί τα σε όλα τα επίπεδα του κατωφλίου και επιτρέπει την εύκολη σύγκριση μεταξύ των ταξινομητών για να έχουμε μία εικόνα ποιος υπερτερεί.

Αν η τιμή του AUC-PR είναι υψηλή τότε ο ταξινομητής είναι αποτελεσματικός ως προς την διατήρηση υψηλής ακρίβειας και ανάκλησης σε διαφορετικά επίπεδα κατωφλίων. Αν η τιμή είναι μικρή τότε ο ταξινομητής δυσκολεύεται είτε να ανιχνεύσει τις θετικές κλάσεις για τα διάφορα επίπεδα του κατωφλίου, είτε να αποφύγει την λανθασμένη ταξινόμηση δεδομένων ως θετικά για τα διάφορα επίπεδα του κατωφλίου.

Οπότε βλέποντας την μορφή της καμπύλης και βλέποντας την τιμή της AUC που υπολογίστηκε συμπεραίνουμε ότι το μοντέλο έχει μία αρκετά καλή συμπεριφορά και μπορεί να κάνει καλές προβλέψεις.

Η καμπύλη Precision-Recall αυτού του μοντέλου δίνεται εδώ:



Τα αποτελέσματα φαίνονται αρκετά καλά και για τις δύο κλάσεις. Οι καμπύλες Precision-Recall δείχνουν ότι η συμπεριφορά του ταξινομητή είναι αρκετά καλή σε γενικές γραμμές και ειδικά για την τιμή του precision που είναι πάντοτε μεγαλύτερη του 0.5. Η καλή συμπεριφορά του ταξινομητή είναι εμφανής και από την τιμή AUC που πλησιάζει αρκετά την τιμή 1 και για τις δύο κλάσεις, οι οποίες είναι και η μέγιστη.

Ερώτημα c

Στα πλαίσια αυτού του ερωτήματος πρέπει να γίνουν προσπάθειες για την βελτίωση της απόδοσης του μοντέλου. Σε πρώτη φάση δεν έγιναν μεγάλες αλλαγές στο προηγούμενο μοντέλο. Η τροποποιημένη εκδοχή του συνελικτικού δικτύου δίνεται παρακάτω:

```
class ModifiedSimpleCNN(nn.Module):
    def __init__(self):
        super(ModifiedSimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2, padding=0)
        self.fc1 = nn.Linear(32 * 16 * 16, 16)
        self.fc2 = nn.Linear(16, 1)

    def forward(self, x):
        x = self.pool(torch.relu(self.bn1(self.conv1(x))))
        x = x.view(-1, 32 * 16 * 16) # Flatten tensor
        x = torch.relu(self.fc1(x))
        x = torch.sigmoid(self.fc2(x))
        return x
```

Οι αλλαγές που εφαρμόστηκαν είναι δύο:

1. Αύξηση του αριθμού των νευρώνων-φίλτρων του συνελικτικού επιπέδου από 8 σε 32.
2. Προσθήκη επιπέδου Batch Normalization

Η πρώτη αλλαγή δίνει την δυνατότητα στο δίκτυο να εξάγει περισσότερα και, ίσως, πιο πολύπλοκα χαρακτηριστικά από τις εικόνες αυξάνοντας σημαντικά την ικανότητα αναγνώρισης του. Η δεύτερη αλλαγή εισάγει ένα επίπεδο το οποίο βοηθάει στην επιτάχυνση της εκπαίδευσης καθώς μπορούν να χρησιμοποιηθούν μεγαλύτερη ρυθμοί μάθησης χωρίς τον κίνδυνο να αυξηθεί η τιμή της απώλειας. Ακόμη η κανονικοποίηση μειώνει την εξάρτηση του δικτύου από

τις αρχικές τιμές των παραμέτρων, σταθεροποιώντας την εκπαίδευση. Τέλος, η κανονικοποίηση βοηθάει στην αποφυγή της υπερπροσαρμογής του μοντέλου στα δεδομένα εκπαίδευσης.

```
modified_simple_model = ModifiedSimpleCNN().to(device)

# Ορισμός της συνάρτησης απώλειας, του βελτιστοποιητή και ρυθμού μάθησης
learning_rate_list = [0.0001, 0.00001]
lr=learning_rate_list[0]
criterion = nn.BCELoss()
optimizer = optim.Adam(modified_simple_model.parameters(), lr=lr)
```

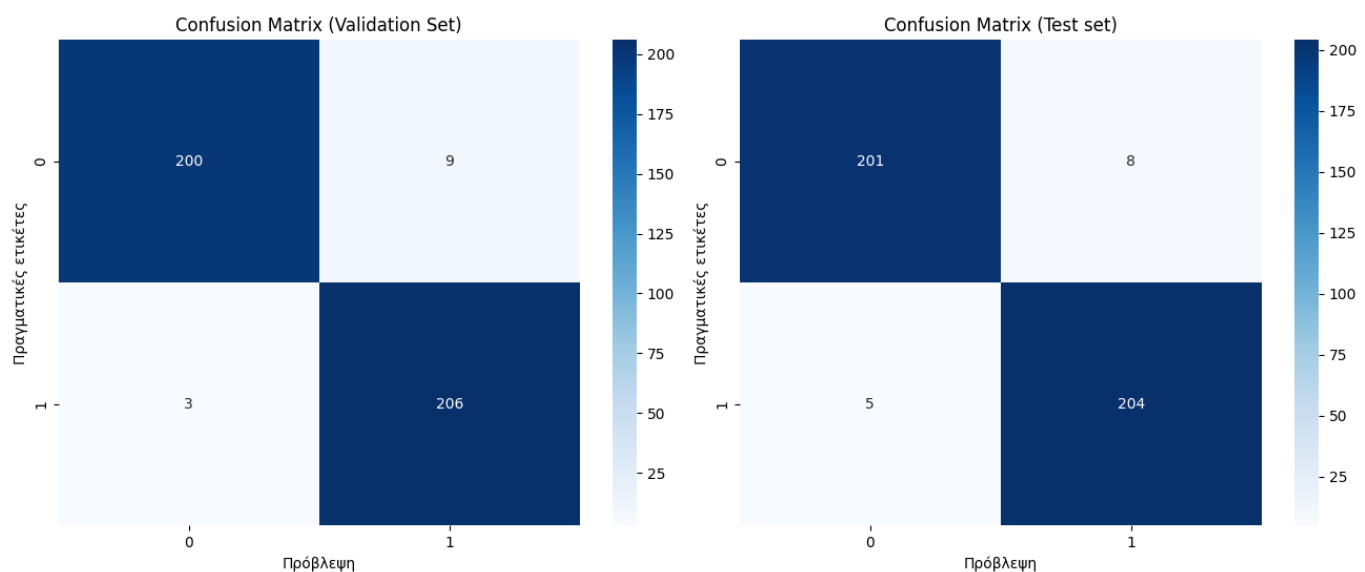
Πέρα από τις παραπάνω αλλαγές, έγινε και μία τρίτη. Ορίστηκε ρυθμός μάθησης αρχικά σε 0.0001, αλλά μετά από 5 εποχές ο ρυθμός μειώνεται σε 0.00001. Οι τιμές του ρυθμού μάθησης ορίζονται σε λίστα και κατά την διάρκεια της εκπαίδευσης μετακινείται στο επόμενο στοιχείο της λίστας.

Στην συνέχεια εκτελείται η εκπαίδευση του νέου μοντέλου που έχει την τροποποιημένη αρχιτεκτονική του αρχικού νευρωνικού δικτύου.

```
Epoch [1/10], Loss: 7.9945, Val Loss: 3.1250, Val Accuracy: 94.02%
Epoch [2/10], Loss: 3.4119, Val Loss: 1.3600, Val Accuracy: 95.22%
Epoch [3/10], Loss: 2.5821, Val Loss: 0.8046, Val Accuracy: 95.93%
Epoch [4/10], Loss: 2.2403, Val Loss: 0.7242, Val Accuracy: 96.41%
Epoch [5/10], Loss: 1.9566, Val Loss: 0.5880, Val Accuracy: 96.89%
Epoch [6/10], Loss: 1.7678, Val Loss: 0.5603, Val Accuracy: 96.89%
Epoch [7/10], Loss: 1.7261, Val Loss: 0.5936, Val Accuracy: 97.13%
Epoch [8/10], Loss: 1.6809, Val Loss: 0.5667, Val Accuracy: 97.13%
Epoch [9/10], Loss: 1.6798, Val Loss: 0.5737, Val Accuracy: 97.13%
Epoch [10/10], Loss: 1.6009, Val Loss: 0.5974, Val Accuracy: 97.13%

Test Loss: 0.76140, Test Accuracy: 96.8900%
```

Βλέπουμε πως το νέο μοντέλο όντως έχει καλύτερη απόδοση τόσο στα δεδομένα του σετ επαλήθευσης, όσο και στα δεδομένα του σετ δοκιμής. Το ίδιο παρατηρείται και στους αντίστοιχους πίνακες σύγχυσης που παράγονται:



Αυτή τη φορά παρατηρείται και στα δύο σετ μία μεγαλύτερη αδυναμία εντοπισμού των πραγματικών δεδομένων που δεν γίνεται χρήση μάσκας. Βέβαια η βελτίωση είναι ιδιαίτερα σημαντική και ο αριθμός των σφαλμάτων είναι πολύ μικρός σε σχέση με τα συνολικά δεδομένα.

Μία πράξη που θα μπορούσε να μετατοπίσει το σφάλμα προς την άλλη πλευρά, πιθανώς χωρίς να μειώσει ιδιαίτερα την απόδοση είναι μετατόπιση του threshold απόφασης από το κέντρο (outputs > 0.5) προς μεγαλύτερη τιμή ώστε να ταξινομεί μία είσοδο δυσκολότερα ως with_mask. Επειδή όμως προς το παρόν η επιθυμία μας είναι μία γενική βελτίωση του συστήματος ταξινόμησης προτίμησα να τροποποιήσω περαιτέρω την αρχιτεκτονική του δικτύου.

Το νέο νευρωνικό δίκτυο στο οποίο κατέληξα μετά από αρκετές δοκιμές είναι αρκετά πολυπλοκότερο και δίνεται παρακάτω:

```
class Complex_LayerCNN(nn.Module):
    def __init__(self):
        super(Complex_LayerCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=5, stride=1, padding=2)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2, padding=0)
        self.dropout = nn.Dropout(0.5)
        self.fc1 = nn.Linear(64 * 8 * 8, 512)
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, 1)

    def forward(self, x):
        x = self.pool(torch.relu(self.bn1(self.conv1(x))))
        x = self.pool(torch.relu(self.bn2(self.conv2(x))))
        x = x.view(-1, 64 * 8 * 8)
        x = self.dropout(torch.relu(self.fc1(x)))
        x = self.dropout(torch.relu(self.fc2(x)))
        x = torch.sigmoid(self.fc3(x))
        return x
```

Το νέο μοντέλο έχει δεύτερο συνελκτικό επίπεδο στο οποίο επίσης εφαρμόζεται Max Pooling και Batch Normalization. Έχει επίσης 2 κρυφά fully connected επίπεδα αντί για μόνο 1, τα οποία έχουν και τα 2 περισσότερους νευρώνες από τα προηγούμενα μοντέλα. Πέρα από τον μεγαλύτερο αριθμό νευρώνων, σε αυτά τα δύο επίπεδα εφαρμόζεται η λειτουργία dropout με πιθανότητα 50% για την αποφυγή εμφάνισης υπερπροσαρμογής.

Ακόμη το πρώτο συνελκτικό επίπεδο έχει περισσότερους νευρώνες (32 αντί για 16) και kernel μεγαλύτερου μεγέθους, το οποίο συνεπάγεται και μεγαλύτερο padding για να μην εφαρμοστεί κάποια μη επιθυμητή μείωση του μεγέθους της εικόνας όταν περάσεις στο επόμενο επίπεδο.

Το μοντέλο που ορίστηκε με αυτή την αρχιτεκτονική έχει ορισμένες τις παρακάτω παραμέτρους για την εκπαίδευσή του:

```
complex_model = Complex_LayerCNN().to(device)

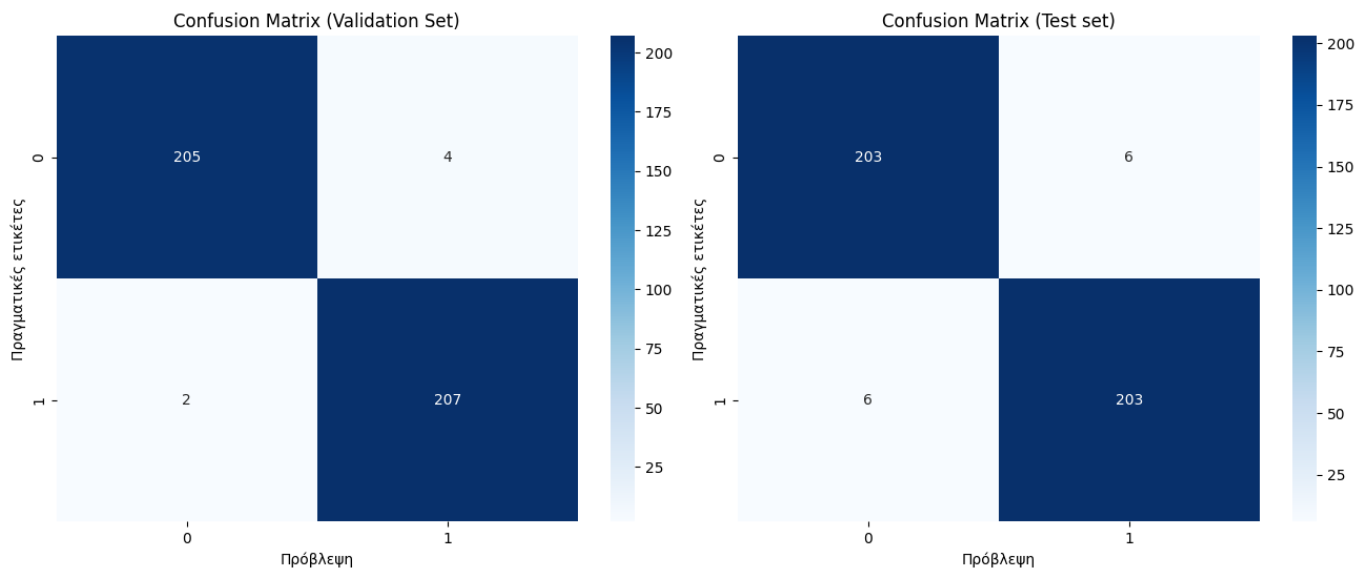
# Ορισμός της συνάρτησης απώλειας, του βελτιστοποιητή και ρυθμού μάθησης
learning_rate_list = [0.0001, 0.00001]
lr=learning_rate_list[0]
criterion = nn.BCELoss()
optimizer = optim.Adam(complex_model.parameters(), lr=lr)

train_model_lr_list(complex_model, train_loader, val_loader, learning_rate_list, criterion, 10, True)
```

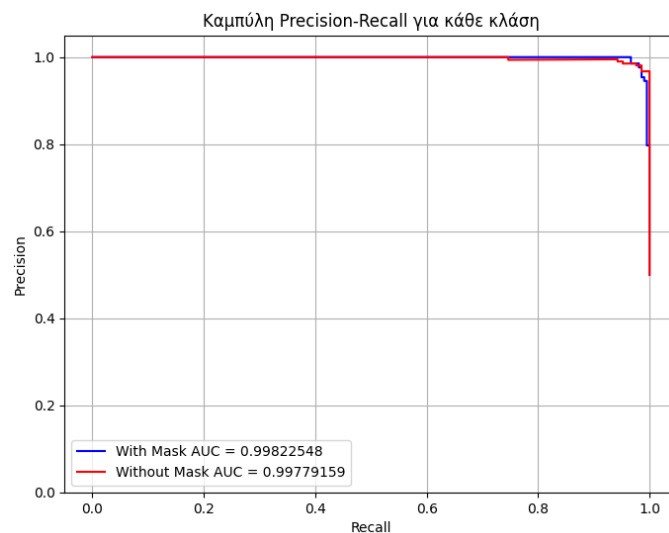
Η εκπαίδευση είχε βελτιωμένα αποτελέσματα, όχι μόνο για την εκπαίδευση που παραδίδω, αλλά γενικότερα μετά από αρκετές εκπαιδεύσεις.

```
Epoch [1/10], Loss: 9.0779, Val Loss: 3.1093, Val Accuracy: 96.17%
Epoch [2/10], Loss: 3.2581, Val Loss: 0.7123, Val Accuracy: 97.13%
Epoch [3/10], Loss: 2.0914, Val Loss: 0.4623, Val Accuracy: 97.85%
Epoch [4/10], Loss: 1.7384, Val Loss: 0.3579, Val Accuracy: 98.56%
Epoch [5/10], Loss: 1.3576, Val Loss: 0.2893, Val Accuracy: 98.56%
Epoch [6/10], Loss: 1.1723, Val Loss: 0.3004, Val Accuracy: 98.56%
Epoch [7/10], Loss: 0.9298, Val Loss: 0.2923, Val Accuracy: 98.33%
Epoch [8/10], Loss: 1.0127, Val Loss: 0.2998, Val Accuracy: 98.56%
Epoch [9/10], Loss: 1.0440, Val Loss: 0.3045, Val Accuracy: 98.56%
Epoch [10/10], Loss: 0.8855, Val Loss: 0.3478, Val Accuracy: 98.56%
Test Loss: 0.63735, Test Accuracy: 97.1292%
```

Ακόμη, δίνονται οι πίνακες σύγχυσης και η καμπύλη Precision-Recall



Οι πίνακες σύγχυσης αποδεικνύουν την εμφανή βελτίωση της ταξινόμησης των δεδομένων τόσο για το σετ επαλήθευσης, όσο και για το σετ δοκιμής.



Η καμπύλη έχει μετακινηθεί ελαφρώς ψηλότερα και πιο αριστερά, ορίζοντας μεγαλύτερο εμβαδό και μία, γενικότερα, ελαφρώς καλύτερη συμπεριφορά του μοντέλου που εκπαιδεύτηκε για την ταξινόμηση των δεδομένων και για τις δύο κλάσεις που ορίζονται από το πρόβλημα.

Αν και τα αποτελέσματα ήταν ήδη αρκετά καλά, όπως και φαίνεται στις τελευταίες 3 απεικονίσεις, εγώ συνέχισα να ψάχνω τρόπους να τα βελτιώσω έστω και λίγο ακόμα.

Μία ιδέα που είχα ήταν η επαύξηση των δεδομένων (Data Augmentation). Αυτό προέκυψε από την γενική ανάγκη των νευρωνικών για μεγάλο αριθμό δεδομένων. Αν, λοιπόν, εφαρμόσουμε κάποιες τροποποιήσεις (Περιστροφή, Αλλαγή φωτεινότητας, αντίθεσης, κορεσμού) με τυχαίο τρόπο στα δεδομένα που έχουμε μπορούμε να τα προσθέσουμε στα αρχικά και με αυτό τον τρόπο να διπλασιάσουμε τα δεδομένα του μοντέλου.

Τα πλεονεκτήματα που μας δίνει αυτή η προ-επεξεργασία των δεδομένων είναι αρκετά και τα σημαντικότερα είναι:

Βελτίωση της γενίκευσης: Η τεχνική δημιουργεί μεγαλύτερη ποικιλία στα δεδομένα εκπαίδευσης δίνοντας την δυνατότητα στο μοντέλο να μάθει να γενικεύει καλά σε νέα δεδομένα που μπορεί να έχουν διαφορετική μορφή από αυτή των αρχικών δεδομένων που μας δίνονται (διαφορετικές συνθήκες φωτισμού, γωνίας λήψης, κλπ.).

Πραγματικές συνθήκες: Οι τροποποιήσεις που μπορεί να εφαρμοστούν στις εικόνες, δίνουν την δυνατότητα προσομοίωσης δεδομένων σε διαφορετικές συνθήκες (φυσικός ή τεχνητός φωτισμός, χαμηλός ή υψηλός φωτισμός, λήψη από άλλη γωνία, διαφορετική εποχή, κλπ.).

Αύξηση των δεδομένων: Σε ορισμένες περιπτώσεις που τα δεδομένα δεν είναι αρκετά, είναι ένας φθηνός τρόπος να δημιουργήσουμε νέα για την βελτίωση στην απόδοσή του.

Αποφυγή ανεπιθύμητων συσχετίσεων: Καθώς τα δεδομένα ενισχύονται με τυχαίες αλλαγές. Με αυτό το τρόπο το μοντέλο είναι λιγότερο πιθανό να μάθει λανθασμένα χαρακτηριστικά και να βασίζεται σε αυτά για την λήψη αποφάσεων.

Πέρα από την εφαρμογή της τεχνικής Data Augmentation μετά από μερικές δοκιμές αύξησα και τις εποχές εκπαίδευσης σε 15.

Συνεπώς ορίζω νέο μοντέλο, το εκπαιδεύω και το αξιολογώ όπως στα προηγούμενα βήματα.

```

complex_model_augm = Complex_LayerCNN().to(device)

# Ορισμός της συνάρτησης απώλειας, του βελτιστοποιητή και ρυθμού μάθησης
learning_rate_list = [0.001, 0.0005, 0.0001]
lr=learning_rate_list[0]
criterion = nn.BCELoss()
optimizer = optim.Adam(complex_model_augm.parameters(), lr=lr)

train_model_lr_list(complex_model_augm, full_train_loader, val_loader, learning_rate_list, criterion, 15, True)

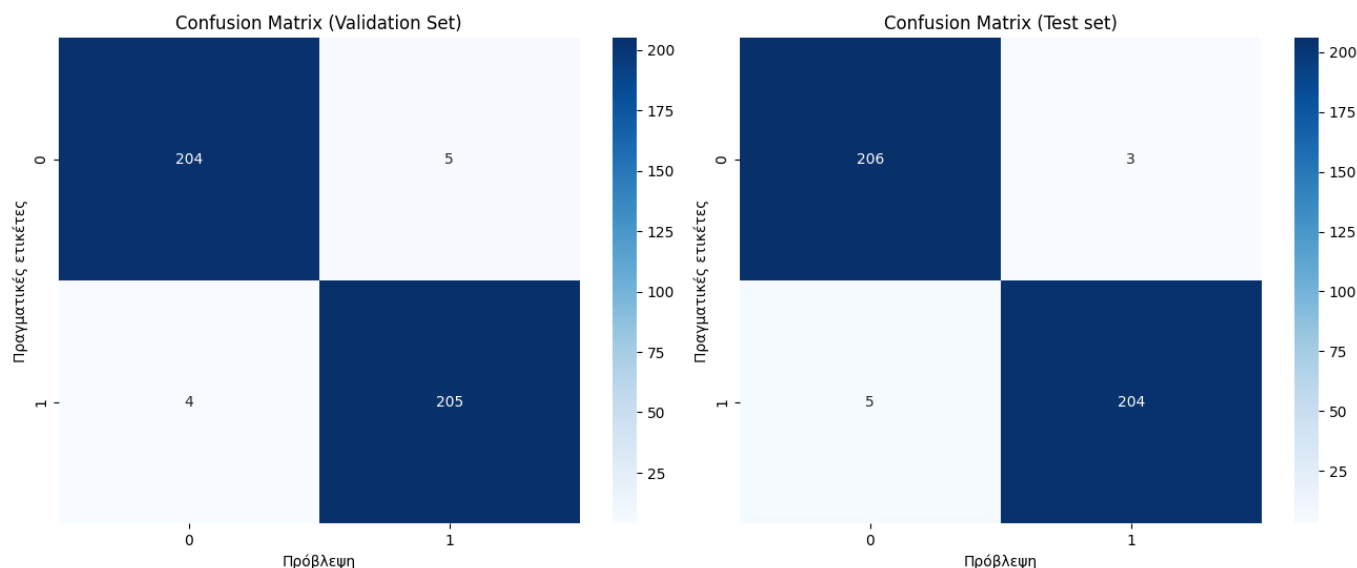
```

```

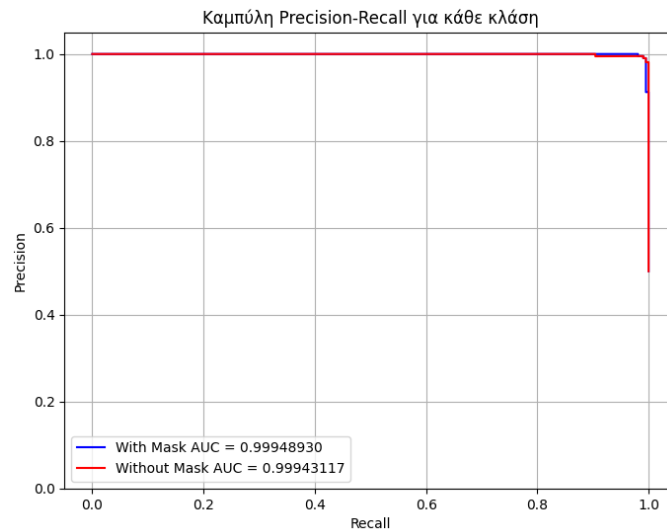
Epoch [1/15], Loss: 11.2071, Val Loss: 0.8187, Val Accuracy: 96.41%
Epoch [2/15], Loss: 4.0984, Val Loss: 1.0428, Val Accuracy: 95.45%
Epoch [3/15], Loss: 3.5000, Val Loss: 0.6587, Val Accuracy: 95.69%
Epoch [4/15], Loss: 2.7032, Val Loss: 2.6538, Val Accuracy: 90.67%
Epoch [5/15], Loss: 3.6531, Val Loss: 0.5872, Val Accuracy: 96.89%
Epoch [6/15], Loss: 2.7236, Val Loss: 0.6180, Val Accuracy: 97.37%
Epoch [7/15], Loss: 1.7118, Val Loss: 0.7007, Val Accuracy: 97.85%
Epoch [8/15], Loss: 1.7823, Val Loss: 0.5980, Val Accuracy: 96.89%
Epoch [9/15], Loss: 1.1980, Val Loss: 0.6279, Val Accuracy: 97.61%
Epoch [10/15], Loss: 1.1451, Val Loss: 0.6009, Val Accuracy: 98.33%
Epoch [11/15], Loss: 0.8344, Val Loss: 0.7475, Val Accuracy: 98.09%
Epoch [12/15], Loss: 0.6483, Val Loss: 0.8746, Val Accuracy: 97.13%
Epoch [13/15], Loss: 0.5813, Val Loss: 0.8674, Val Accuracy: 97.37%
Epoch [14/15], Loss: 0.5497, Val Loss: 0.8558, Val Accuracy: 97.85%
Epoch [15/15], Loss: 0.6237, Val Loss: 0.8328, Val Accuracy: 97.85%
Test Loss: 0.77259, Test Accuracy: 98.0861%

```

Το μοντέλο μετά από αρκετές εκπαιδεύσεις παρατηρήθηκε ότι έχει μία ελαφρά βελτίωση της τάξης του 0.15% κατά μέσο όρο σε σχέση με το ίδιο μοντέλο που δεν εκπαιδεύτηκε σε επαυξημένα δεδομένα. Η βελτίωση δεν είναι ιδιαίτερα μεγάλη, αλλά δεν έχω λόγο να την αγνοήσω.



Εδώ δεν φαίνεται καθαρά η βελτίωση αλλά σε γενικές γραμμές το μοντέλο εφαρμόζει καλή ταξινόμηση στα δεδομένα με σχεδόν μοιρασμένα λάθη στις δύο κατηγορίες. Ίσως έχει λίγο καλύτερη απόδοση στον εντοπισμό της μη χρήσης μάσκας (γενικότερα μετά και από άλλες εκπαιδεύσεις και αξιολογήσεις που είχαν γίνει).



Εδώ είναι ακόμα πιο φανερή η καλή λειτουργία του μοντέλου, διατηρώντας μεγάλες τιμές ταυτόχρονα και για το Precision και για το Recall, πράγμα επιθυμητό.

Πριν προχωρήσω στο ερώτημα d, από προσωπική περιέργεια, αποφάσισα να δοκιμάσω να χρησιμοποιήσω τα δεδομένα εκπαίδευσης που έχουμε (τα επαυξημένα) για να εκπαιδεύσω ένα προ-εκπαιδευμένο μοντέλο σε αυτά και να την απόδοση του.

```
# Φόρτωση προεκπαιδευμένου ResNet-18
resnet18 = models.resnet18(pretrained=True)

# Αλλαγή του τελευταίου γραμμικού επιπέδου για να ταιριάζει με τον αριθμό των κλάσεων
num_of_classes = resnet18.fc.in_features
resnet18.fc = nn.Linear(num_of_classes, 1)

resnet18 = resnet18.to(device)

# Ορισμός της συνάρτησης απώλειας και του βελτιστοποιητή
lr=0.00005
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(resnet18.parameters(), lr=lr)

# Εκτέλεση της εκπαίδευσης
train_model_constant_lr(resnet18, full_train_loader, val_loader, criterion, optimizer, 8, True)

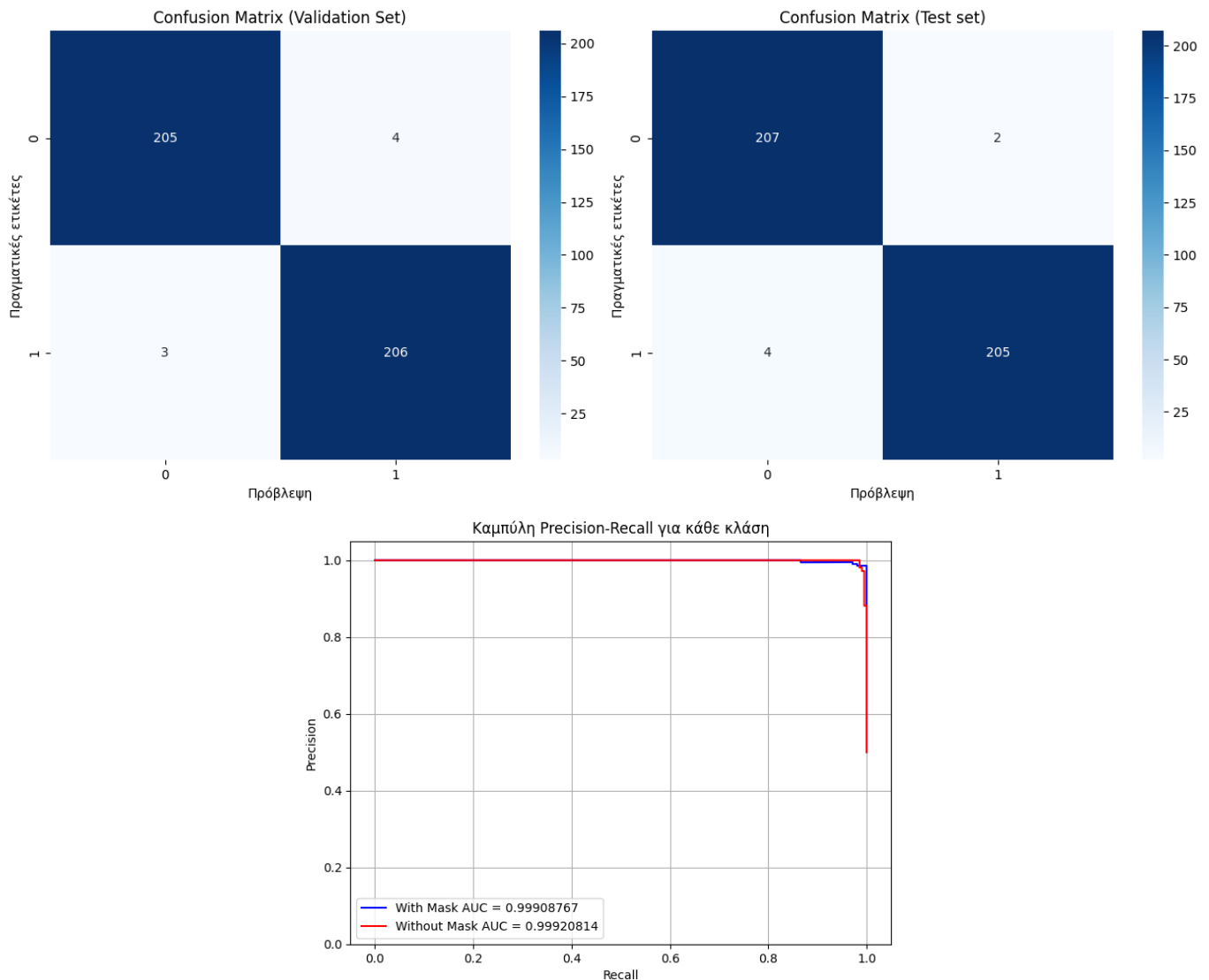
model_evaluation(resnet18, test_loader, criterion, True, True)
```

Το μόνο διαφορετικό που εφαρμόζεται σε αυτό το μοντέλο είναι το κριτήριο (συνάρτηση απώλειας) καθώς στην αρχιτεκτονική του συγκεκριμένου δικτύου δεν έχει οριστεί σιγμοειδής συνάρτηση ενεργοποίησης για το επίπεδο εξόδου. Η συνάρτηση BCEWithLogitsLoss στην ουσία εφαρμόζει την σιγμοειδή στην έξοδο του output layer και στην συνέχεια εφαρμόζει την BCELoss για τον υπολογισμό της απώλειας.

Τα αποτελέσματα είναι τα παρακάτω:

```
Epoch [1/8], Loss: 15.5239, Val Loss: 1.1865, Val Accuracy: 94.50%
Epoch [2/8], Loss: 4.2284, Val Loss: 0.6746, Val Accuracy: 97.13%
Epoch [3/8], Loss: 2.4990, Val Loss: 0.6124, Val Accuracy: 97.37%
Epoch [4/8], Loss: 1.4363, Val Loss: 0.5377, Val Accuracy: 97.13%
Epoch [5/8], Loss: 2.5973, Val Loss: 0.5882, Val Accuracy: 96.65%
Epoch [6/8], Loss: 1.2240, Val Loss: 0.4803, Val Accuracy: 97.13%
Epoch [7/8], Loss: 0.8595, Val Loss: 0.4499, Val Accuracy: 97.13%
Epoch [8/8], Loss: 0.8466, Val Loss: 0.4161, Val Accuracy: 98.33%
```


Test Loss: 0.39460, Test Accuracy: 98.5646%



Όπως φαίνεται από την εκπαίδευση και την αξιολόγηση του μοντέλου τα αποτελέσματα του είναι πολύ καλά. Μάλιστα αρκετές φορές ξεπερνάνε τα αποτελέσματα του τελικού μοντέλου που σχεδίασα και έστησα μόνος μου μετά από αρκετές προσπάθειες.

Σε ένα πραγματικό σενάριο συνεπώς το πρώτο πράγμα που θα δοκίμαζα θα ήταν η χρήση ενός προεκπαιδευμένου μοντέλου για το πρόβλημα μας. Επειδή όμως πρόκειται για μία ακαδημαϊκή εργασία επέλεξα να έχω μία μεγαλύτερη τριβή με το πρόβλημα και να δοκιμάσω ένα προεκπαιδευμένο μοντέλο μόνο από περιέργεια.

Για αυτόν τον λόγο στην συνέχεια της εργασίας θα προχωρήσω με το `complex_model_augm` το οποίο είναι το μοντέλο που έχει την αρχιτεκτονική συνελκτικού νευρωνικού δικτύου `Comple_LayerCNN` που σχεδίασα και που εκπαιδεύτηκε πάνω στα επαυξημένα δεδομένα εκπαίδευσης.

Ερώτημα d

Σε αυτό το ερώτημα αρχικά εισάγω τα δεδομένα του συνόλου `“incorrect_mask_use”` και τα εφαρμόζω, σε πρώτη φάση μόνον τους, στο μοντέλο για να δω την απόδοση του σε τέτοιες περιπτώσεις.

```

1 # Εφαρμογή του μοντέλου στις εικόνες της "mask_incorrect_use"
2 complex_model_augm.eval()
3 correctly_classified_as_without_mask = 0
4 total_incorrect_mask_use = len(incorrect_mask_loader.dataset)
5
6 with torch.no_grad():
7     for images, _ in incorrect_mask_loader:
8         outputs = complex_model_augm(images)
9         predicted = (outputs > 0.5).float()
10        correctly_classified_as_without_mask += (predicted == 0).sum().item()
11
12 accuracy_incorrect_use_as_without_mask = 100 * correctly_classified_as_without_mask / total_incorrect_mask_use
13 print(f'Ποσοστό των δεδομένων της "mask_incorrect_use" που ταξινομούνται (ορθά) ως "without_mask": {accuracy_incorrect_use_as_without_mask:.4f}%')

```

Ποσοστό των δεδομένων της "mask_incorrect_use" που ταξινομούνται (ορθά) ως "without_mask": 19.6429%

Βλέποντας τα αποτελέσματα, αν και χαμηλά, ήταν υψηλότερα από όσο περίμενα. Για την βελτίωσή τους δύο σενάρια που λειτουργούν βρέθηκαν.

Η βελτίωση που θα γίνει στο μοντέλο θα γίνει σε 3 στάδια:

1. Βελτίωση του μοντέλου αποκλειστικά στα δεδομένα του συνόλου "incorrect_mask_use". Θέλουμε το μοντέλο να μπορεί να λειτουργήσει επαρκώς και για δεδομένα που βρίσκονται οπτικά (σαν εικόνα) σε μία ενδιάμεση κατάσταση.
2. Βελτίωση του μοντέλου τόσο για τα προβληματικά δεδομένα όσο και για τα δεδομένα με ετικέτα 0 (without_mask) στο σύνολο δοκιμής.
3. Βελτίωση του μοντέλου ώστε να λειτουργεί επαρκώς καλά για όλα τα δεδομένα.

Βελτίωση 1

Το πρώτο σενάριο ήταν η εφαρμογή τυχαίων τροποποιήσεων στις εικόνες πριν εισαχθούν στο μοντέλο για να τις ταξινομήσει. Η τροποποιήσεις είναι σαν αυτές που έγιναν για την επαύξηση των δεδομένων εκπαίδευσης. Για να ελεγχθεί η βελτίωση κατά μέσο όρο του μοντέλου οι τροποποιήσεις έγιναν επαναληπτικά 100 φορές και τα νέα δεδομένα κάθε φορά εισάγονται για αξιολόγηση στο μοντέλο.

Οι τροποποιήσεις αφορούν περιστροφή των εικόνων έως και 90° και αυξομείωση της φωτεινότητας και της αντίθεσης έως και 50%. Αυτές οι τροποποιήσεις έγιναν με τυχαίο τρόπο. Τα αποτελέσματα ήταν σημαντική αύξηση, κατά μέσο όρο, στην ακρίβεια του μοντέλου.

Μέση ακρίβεια 29.910714285714285%

Η δεύτερη τροποποίηση αφορά την μετακίνηση του επιπέδου απόφασης υπέρ της κλάσης without_mask. Στην ουσία ορίζουμε το μοντέλο να ταξινομεί ένα σενάριο ως with_mask μόνο αν είναι πολύ «σίγουρο». Το κατώφλι αρχικά ήταν στο 50% (0.5), αλλά το μετακινήσαμε στο 95% (0.95). Τα αποτελέσματα, και πάλι μετά από αρκετές επαναλήψεις, είναι τα παρακάτω:

Μέση ακρίβεια 42.9286%

Όπως φαίνεται η βελτίωση του μοντέλου έχει αυξηθεί σε πολύ μεγάλο βαθμό. Θα μπορούσαμε να μετακινήσουμε ακόμα περισσότερο, αλλά μέχρι αυτό το σημείο φαίνεται πολύ ακραία η μετακίνηση που έχει γίνει ήδη στο 95%.

Το ζήτημα είναι όμως ότι αν η βελτίωση αυτή χειροτερεύει κατά πολύ τα αποτελέσματα πάνω στα «κανονικά» δεδομένα τότε πρέπει να περιοριστεί. Αυτά θα τα δούμε στο δεύτερο στάδιο βελτίωσης.

Βελτίωση 2

Εδώ συνδυάζουμε τα δεδομένα των συνόλων του τεστ σετ μαζί με τα δεδομένα του συνόλου incorrect_mask_use. Η αξιολόγηση όμως θα γίνει μόνο για τα δεδομένα που πρέπει να ταξινομηθούν ως "without_mask", για τις τροποποιήσεις που εφαρμόστηκαν στο προηγούμενο στάδιο. Τα αποτελέσματα είναι:

Μέση ακρίβεια 83.5177%

Αν δεν εφαρμόσουμε καμία τροποποίηση πριν τα εισάγουμε τις εικόνες στο μοντέλο για ταξινόμηση (διατηρώντας όμως το κατώφλι στο 95%) είναι τα παρακάτω:

Μέση ακρίβεια 83.6879%

Εν τέλει κατέληξα να εφαρμόσω τροποποιήσεις στα δεδομένα πριν την εισαγωγή τους στο μοντέλο, αλλά πολύ μικρότερες από πριν. (Περιστροφή μέχρι 20° και αυξομείωση της φωτεινότητας και της αντίθεσης μόνο μέχρι 20%) Τα αποτελέσματα:

Μέση ακρίβεια 83.8298%

Δεν εφαρμόστηκε κάποια μετατόπιση στο κατώφλι γιατί αν καταστρέψουμε την ικανότητα χαρακτηρισμού δεδομένων ως `with_mask` δεν θα έχει καθόλου νόημα το μοντέλο σαν σύστημα ταξινόμησης. Για να εφαρμόσουμε περαιτέρω αλλαγές στο κατώφλι (αν μπορούμε) χρειάζεται να το κάνουμε ελέγχοντας όλα τα δεδομένα προς αξιολόγηση.

Βελτίωση 3

Εκτελώ αξιολόγηση των δεδομένων χρησιμοποιώντας όλα τα δεδομένα αυτή τη φορά. Μία φορά χωρίς καμία από τις βελτιώσεις.

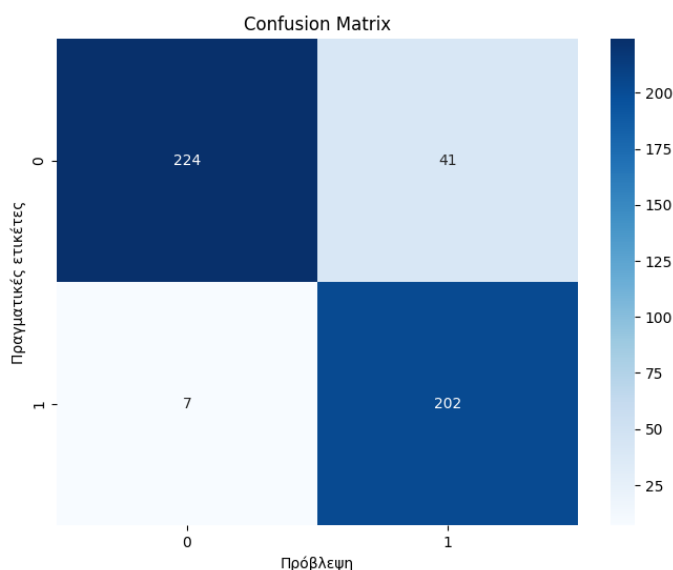
Μέση ακρίβεια 82.2695%

Μπορούμε να πούμε ότι γενικά το μοντέλο που υλοποιήσαμε στο ερώτημα c δεν λειτουργεί ιδιαίτερα καλά όταν εισάγουμε ενδιάμεσες καταστάσεις.

Εν τέλει κατέληξα στο γεγονός ότι το μοντέλο δεν λειτουργεί καλά αν γίνει επαύξηση στα δεδομένα. Οπότε εκτέλεσα την αξιολόγηση χωρίς επαύξηση για το κατώφλι όμως στην τιμή 0.95 (95%). Τα αποτελέσματα ήταν:

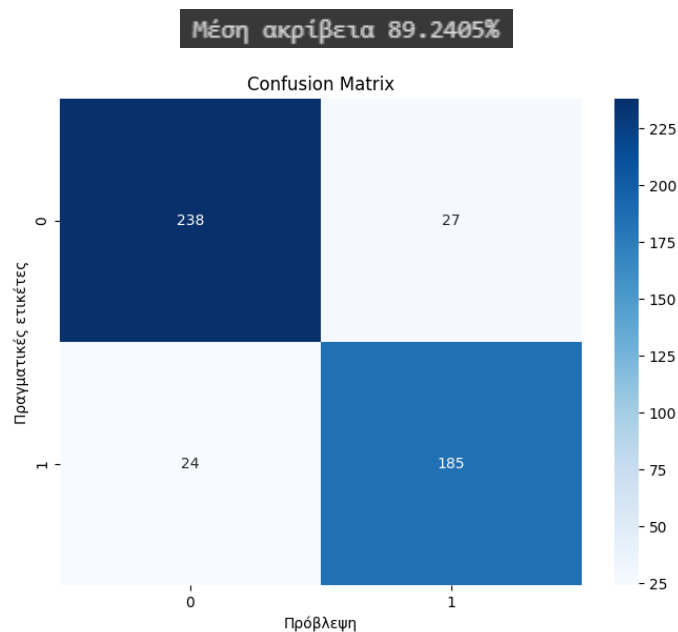
Μέση ακρίβεια 89.8734%

Όμως βλέποντας την ταξινόμηση των δεδομένων από τον πίνακα σύγχυσης, τα αποτελέσματα είναι δυσάρεστα.



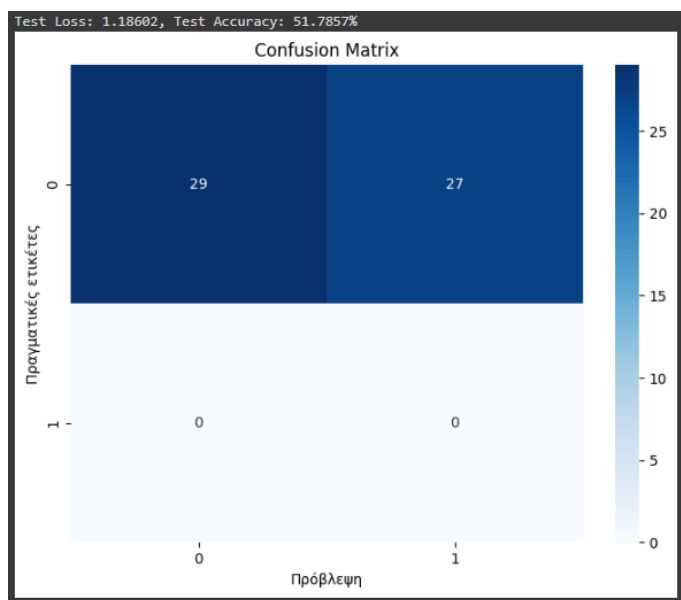
Όπως φαίνεται το μοντέλο είναι ιδιαίτερα καλό στην ταξινόμηση των δεδομένων σωστής χρήσης μάσκα αλλά αδυνατεί να ανταπεξέλθει στα δεδομένα μη χρήσης μάσκα και μάλλον περισσότερο σε αυτά με την λανθασμένη χρήση της.

Βλέπουμε ότι αν αυξήσουμε το κατώφλι για την απόφαση μοντέλου αρκετά περισσότερο (στο 0.999998 ή 99.9998%) μπορούμε να διατηρήσουμε την ακρίβεια του μοντέλου μοιράζοντας το σφάλμα καλύτερα μεταξύ των δύο κατηγοριών .



Για την τελική μορφή του μοντέλου έκανα δύο εκτελέσεις ακόμα.

1) Μόνο για τα δεδομένα `incorrect_mask_use`:



Η απόδοση του αρχικού μοντέλο ήταν 19% και πλέον είναι σχεδόν 52%. Η βελτίωση αυτή είναι πολύ σημαντική.

2) Για όλα τα δεδομένα με ετικέτα 0:

Μέση ακρίβεια 85.1064%

Μέρος 2 - Πρόβλεψη πρόσδεσης χημικών μορίων σε βιολογικούς υποδοχείς

Εκφώνηση:

Στο δεύτερο μέρος καλείστε να επιλύσετε ένα πρόβλημα πρόβλεψης του βαθμού πρόσδεσης (binding) χημικών μορίων σε βιολογικούς υποδοχείς. Η διαδικασία αυτή χρησιμοποιείται συχνά στη φαρμακολογία και τη βιοχημεία για την ανακάλυψη χημικών μορίων που δυνητικά μπορούν να αποτελέσουν ενεργές ουσίες που προσδένονται σε κάποιο υποδοχέα-στόχο και αναστέλλουν τη δράση του. Στο αρχείο `Data_Receptors.zip` θα βρείτε δεδομένα εκπαίδευσης και δοκιμής για 1115 και 124 χημικά μόρια αντίστοιχα, για καθένα από τα οποία έχουν υπολογισθεί 3473 χαρακτηριστικά. Τα πρώτα 1425 χαρακτηριστικά (κατά στήλες) μπορούν να λάβουν συνεχείς τιμές, ενώ τα 2048 επόμενα αποτελούν binary περιγραφείς των μορίων. Για τα δεδομένα εκπαίδευσης σας δίνονται και οι ετικέτες που καθορίζουν εάν το κάθε μόριο μπορεί να προσδεθεί αποτελεσματικά στον υποδοχέα στόχο (label 1) ή όχι (label 0).

- Σχεδιάστε και υλοποιήστε ένα μοντέλο μηχανικής μάθησης χρησιμοποιώντας κατάλληλους αλγορίθμους της επιλογής σας, συμβατικούς ή/και νευρωνικούς, που δέχεται σαν είσοδο τα χαρακτηριστικά νέων χημικών μορίων και προβλέπει αν το κάθε μόριο μπορεί να συνδεθεί στον υποδοχέα στόχο. Δικαιολογήστε το σκεπτικό σας.
- Τα διαθέσιμα δεδομένα για βελτιστοποίηση και εκπαίδευση του μοντέλου σας είναι μονάχα αυτά που περιλαμβάνονται στο αρχείο `Train_features.csv` για τα οποία έχετε διαθέσιμες ετικέτες (`Train_labels.csv`). Ακολουθήστε το κατάλληλο πρωτόκολλο ώστε να βελτιστοποιήσετε τις παραμέτρους του μοντέλου σας, και να εκτιμήσετε με τον καλύτερο δυνατό τρόπο το πιθανό σφάλμα στο test set.
- Δώστε την εκτίμησή σας για το σφάλμα που θα έχει το εκπαιδευμένο μοντέλο σας στο test set, δεδομένου ότι η κατανομή κλάσεων στο test set είναι παρόμοια με αυτή των δεδομένων εκπαίδευσης. Να εκτιμήσετε επίσης το AUC για την καμπύλη TP-FP. Τέλος, να παράξετε ένα αρχείο με όνομα `test_predictions.csv` στο οποίο θα καταχωρήσετε την πρόβλεψη του μοντέλου σας για κάθε δείγμα από το test set στη μορφή `predicted_label,prediction_score` σε κάθε γραμμή, με τη σειρά που περιέχονται τα δεδομένα στο `test_features.csv`. Το `prediction_score` αποτελεί την πρωτογενή έξοδο του ταξινομητή σας και πρέπει να είναι είτε η πιθανότητα πρόσδεσης είτε να έχει αύξουσα τιμή για αυξανόμενη πιθανότητα πρόσδεσης. Το αρχείο να συμπεριληφθεί μαζί με τα υπόλοιπα παραδοτέα της εργασίας στην υποβολή σας.

Επίλυση:

Σε πρώτη φάση εισάγουμε τα δεδομένα και πρέπει να δούμε αν χρειάζονται κάποια προ-επεξεργασία και να την εφαρμόσουμε.

Είναι εξ αρχής φανερό ότι τα δεδομένα του προβλήματος ανήκουν σε έναν χώρο πάρα πολλών διαστάσεων. Πράγμα που μπορεί να δημιουργήσει προβλήματα. Το πρώτο και σημαντικότερο πρόβλημα είναι η απαίτηση σε πόρους που χρειάζεται για να γίνει η εκπαίδευση ενός ταξινομητή πάνω σε αυτά. Επίσης ο μεγάλος αυτός χώρος σε διαστάσεις μπορεί να οδηγήσει σε υπερπροσαρμογή του συστήματος ταξινόμησης, μη επιτρέποντας την γενίκευση σε νέα δεδομένα. Τέλος όταν οι διαστάσεις των δεδομένων είναι μεγαλύτερες από τον αριθμό τους, τα δεδομένα τείνουν να φαίνονται πιο ομοιογενή, πράγμα που δυσκολεύει την διάκριση τους για να ταξινομηθούν.

Ένα ακόμα ζήτημα που παρατηρείται στα δεδομένα είναι η κλίμακα μέτρησης κάθε χαρακτηριστικού. Είναι φανερό πως αυτό μπορεί να οδηγήσει σε λάθος εκτίμηση για το πιο χαρακτηριστικό είναι σημαντικότερο από ένα άλλο. Αυτό μπορεί να λυθεί εύκολα με κανονικοποίηση των δεδομένων, πριν προχωρήσουμε σε οποιαδήποτε άλλη κίνηση επεξεργασίας των δεδομένων.

Με έναν έλεγχο μάλιστα παρατηρήθηκε ότι κάποιες στήλες είναι είτε σταθερές, είτε και διπλότυπες. Αυτές οι στήλες δεν δίνουν πληροφορία και ίσως θα έπρεπε να αφαιρεθούν τελείως. Αυτό θα μείωνε τις διαστάσεις των δεδομένων χωρίς να χαθεί ούτε ελάχιστη πληροφορία από αυτά, η οποία να μας είναι χρήσιμη.

Για να μην γίνει όμως χειροκίνητα η εργασία αφαίρεσης χαρακτηριστικών (στηλών) που δεν μας χρειάζονται αποφάσισα να χρησιμοποιήσω μία μέθοδο μείωσης διαστάσεων που με βάσει το επιθυμητό variance που θα επιλέξω να κρατήσω, θα ξεφορτωθεί τις παραπάνω στήλες (σταθερές και διπλότυπες), αλλά και χαρακτηριστικά που εξαρτώνται μεταξύ τους θα τα συνδυάσει σε λιγότερες κύριες συνιστώσες.

Αφού έγιναν όλες αυτές οι παρατηρήσεις ξεκίνησα εισάγοντας τα δεδομένα στο αρχείο Notebook που θα γίνει η υλοποίηση της επίλυσης της άσκησης.

Στην συνέχεια πριν κάνω οτιδήποτε άλλο χωρίζω το σετ σε 2 μέρη. Ένα για την εκπαίδευση και ένα για να χρησιμοποιηθεί εν μέρει σαν σετ επαλήθευσης (το οποίο θα έχει ίδιο μέγεθος με το σετ που μας δίνεται χωρίς ετικέτες) και ότι περισσέψει ως σετ δοκιμής.

Συνεπώς ο πρώτος διαχωρισμός θα γίνει με αναλογία 85/15, ενώ ο δεύτερος θα γίνει έτσι ώστε το σετ επαλήθευσης να έχει μέγεθος 124, δηλαδή το σετ δοκιμής να έχει μέγεθος $\text{int}(0.15 * 1115 - 124)$.

Οπότε τα τελικά μεγέθη που προκύπτουν είναι:

Μέγεθος σετ εκπαίδευσης: 947 Δεδομένα

Μέγεθος σετ επαλήθευσης: 124 Δεδομένα

Μέγεθος σετ δοκιμής: 44 Δεδομένα

Στην συνέχεια εφαρμόζω κανονικοποίηση στο σετ εκπαίδευσης και αφού υπολογίσω τις απαραίτητες τιμές (μέση τιμή και τυπική απόκλιση) από το σετ εκπαίδευσης εφαρμόζω την ίδια κανονικοποίηση και στα άλλα 2 σετ.

Η μέθοδος μείωσης διαστάσεων που επέλεξα είναι η PCA (Ανάλυση Κύριων Συνιστωσών) και υπολόγισα για διάφορα ποσοστά διατήρησης του αρχικού variance τον αριθμός συνιστωσών που είναι απαραίτητα για την εξήγηση του.

Πριν εφαρμόσω την μέθοδο PCA κανονικοποίησα τα δεδομένα (όσα δεν είχαν δυαδική τιμή μέτρησης) που μου δόθηκαν γιατί τα αρχικά δεδομένα έχουν διαφορετικές κλίμακες για την μέτρηση διαφορετικών χαρακτηριστικών, κάτι που μπορεί να οδηγήσει την PCA, να θεωρήσει σημαντικά κάποια χαρακτηριστικά μικρότερης σημασίας.

Ποσοστό εξήγησης της αρχικής διασποράς	Κύριες συνιστώσες που απαιτούνται (μετά από κανονικοποίηση)	
50%	7	99.80% ↓
60%	12	99.65% ↓
70%	21	99.40% ↓
80%	39	98.88% ↓
85%	58	98.33% ↓
90%	89	97.44% ↓
95%	156	95.51% ↓
96%	181	94.79% ↓
97%	215	93.81% ↓
98%	264	92.40% ↓
98.5%	299	91.39% ↓
99%	347	90.00% ↓
99.5%	426	87.73% ↓
99.9%	574	83.47% ↓
99.99%	702	79.79% ↓
100%	948	72.70% ↓

Έχοντας αυτά τα αποτελέσματα υπόψη, αποφασίστηκε να προχωρήσουμε με την διατήρηση 426 κύριων συνιστωσών μειώνοντας σχεδόν κατά 88% τις διαστάσεις του προβλήματος, αλλά διατηρώντας το 99.5% της διασποράς του σετ δεδομένων στον αρχικό χώρο των 3473 διαστάσεων.

Αξίζει να αναφερθεί ότι για την εξήγηση σε ποσοστό 100% της αρχικής διασποράς, μπορούμε να κρατήσουμε 948 κύριες συνιστώσες μειώνοντας και πάλι σε μεγάλο βαθμό τις διαστάσεις του προβλήματος, συγκεκριμένα σε ποσοστό περίπου 73%. Παρόλα αυτά θεωρήθηκε και πάλι μεγάλος ο αριθμός διαστάσεων για αυτό επιλέχθηκε να θυσιαστεί ένα μικρό ποσοστό εξήγησης της αρχικής διασποράς.

Η μείωση διαστάσεων αποτελεί, ένα καθοριστικό βήμα για την επίλυση της άσκησης, γιατί παρά την απώλεια πληροφορίας, η οποία πρακτικά είναι μηδαμινή (0.5%), μειώνεται σε τεράστιο βαθμό η πολυπλοκότητα του προβλήματος. Αυτό σημαίνει ότι οι υπολογιστικοί πόροι που απαιτούνται για να γίνει η εκπαίδευση του μοντέλου είναι πολύ λιγότεροι από ότι αν χρησιμοποιούσαμε το αρχικό σετ δεδομένων, κάτι που συνεπάγεται και σε σημαντική μείωση του χρόνου εκπαίδευσης του ταξινομητή.

Δεν θα μείνουμε όμως εδώ. Η PCA είναι μία μέθοδος μείωσης διαστάσεων με την ελάχιστη δυνατή απώλεια της διασποράς των δεδομένων. Αποτελεί όμως μία unsupervised μέθοδο και δεν λαμβάνει υπόψη τις ετικέτες των δεδομένων, τις οποίες και έχουμε στην διάθεσή μας.

Μπορούμε να εφαρμόσουμε στην συνέχεια την μέθοδο LDA (Linear Discriminant Analysis) η οποία λαμβάνει υπόψη τις ετικέτες και κατά την μείωση των διαστάσεων επιχειρεί να διαχωρίσει τις κλάσεις μεταξύ τους και να συμπτυνώσει τα δεδομένα κάθε κλάσης εσωτερικά της κάθε μιας.

Ο λόγος που εκτελέστηκε πρώτα η PCA πάνω στα δεδομένα εκπαίδευσης και στην συνέχεια η μέθοδος LDA δίνονται παρακάτω:

Μείωση θορύβου: Η PCA μπορεί να αφαιρέσει τις διαστάσεις με χαμηλή διακύμανση, οι οποίες συνήθως περιέχουν θόρυβο ή πληροφορίες χωρίς ιδιαίτερη σημασία.

Προστατεύει από υπερπροσαρμογή: Η PCA μπορεί να μειώσει τον αριθμό των χαρακτηριστικών (διαστάσεων του προβλήματος) μειώνοντας την πολυπλοκότητα του προβλήματος. Αυτό μπορεί να μειώσει σημαντικά τον κίνδυνο εμφάνισης του φαινομένου overfitting.

Διαχείριση μεγάλων διαστάσεων: Σε περιπτώσεις όπως αυτή, που τα δεδομένα έχουν πολύ μεγάλες διαστάσεις η PCA μπορεί να μειώσει τον αριθμό αυτών σημαντικά, ώστε να γίνει πιο διαχειρίσιμος από την τεχνική LDA. Δεν πρέπει να ξεχνάμε ότι οι κλάσεις του προβλήματος είναι 2 και συνεπώς η LDA θα μεταφέρει τα δεδομένα σε έναν χώρο μόνο μίας διάστασης.

Συνεπώς μετά από αυτούς τους μετασχηματισμούς τα δεδομένα πλέον είναι αρκετά πιο απλά και οι απαραίτητοι πόροι για την εκτέλεση των υπολογισμών είναι ακραία λιγότεροι.

Κατόπιν σκέψης, περιόρισα τις επιλογές μου μεταξύ ενός ταξινομητή τύπου Random Forest και ενός νευρωνικού δικτύου. Ο Random Forest, αν τον επέλεγα εν τέλει, θα ήταν γιατί παρατηρήθηκε να έχει πολύ καλή ακρίβεια κατά την υλοποίηση του στα πλαίσια της εργασίας 4.

Εν τέλει αποφάσισα να δημιουργήσω διαφορετικού τύπου ταξινομητές και να επιλέξω αυτόν που έχει την καλύτερη επίδοση. Οι ταξινομητές που επέλεξα να υλοποιήσω είναι:

1. Νευρωνικό Δίκτυο
2. Ταξινομητής SVM
3. Συνδυασμός των δύο μοντέλων

Νευρωνικό Δίκτυο

Αρχικά δημιουργήθηκε ένα απλό μοντέλο νευρωνικού δικτύου στο οποίο εφάρμοσα παραμέτρους, τις οποίες δοκιμάζω σταθερά σαν πρώτες επιλογές.

```
class TestNeuralNetwork(nn.Module):
    def __init__(self):
        super(TestNeuralNetwork, self).__init__()
        self.fc1 = nn.Linear(data_train_lda_df.shape[1], 64)
        self.fc2 = nn.Linear(64, 32)
        self.fc3 = nn.Linear(32, 1)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = torch.sigmoid(self.fc3(x))
        return x

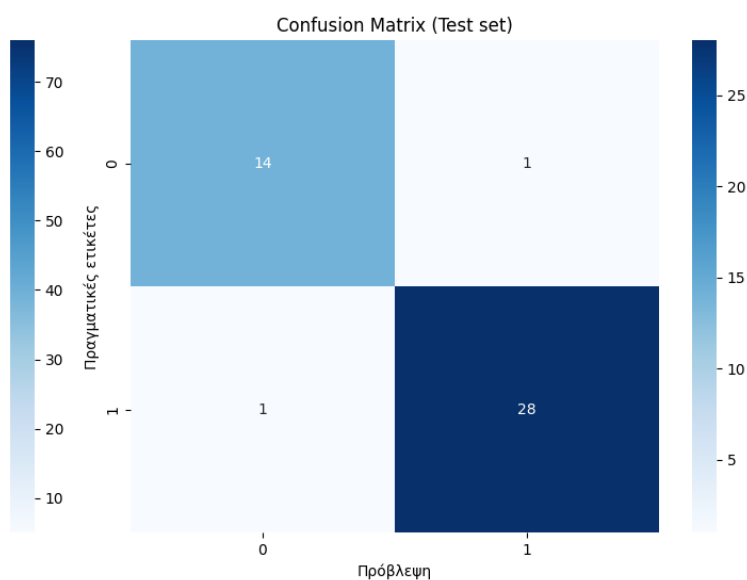
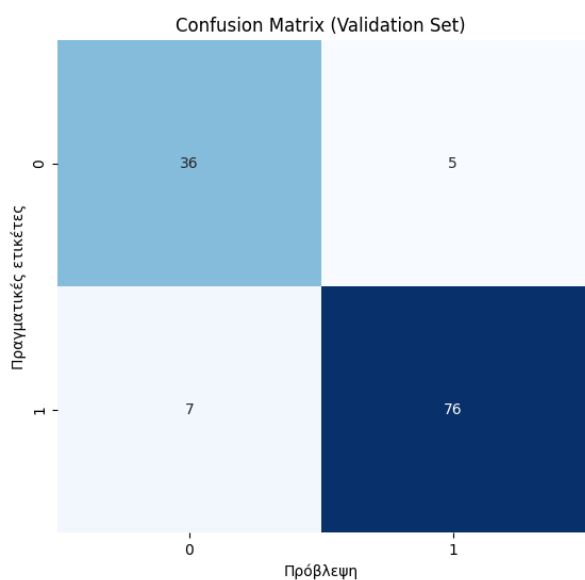
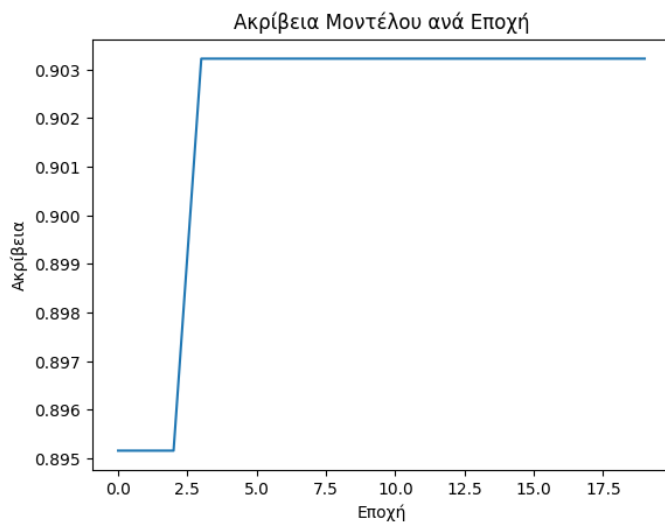
model_1 = TestNeuralNetwork()

# Συνάρτηση Απώλειας και Βελτιστοποιητής
criterion = nn.BCELoss()
learning_rate = 0.001
optimizer = optim.Adam(model_1.parameters(), lr=learning_rate)
```

Και εκτελώ εκπαίδευση πάνω στα δεδομένα εκπαίδευσης και στην συνέχεια αξιολογείται το μοντέλο πάνω στο μικρότερο σετ δοκιμής.

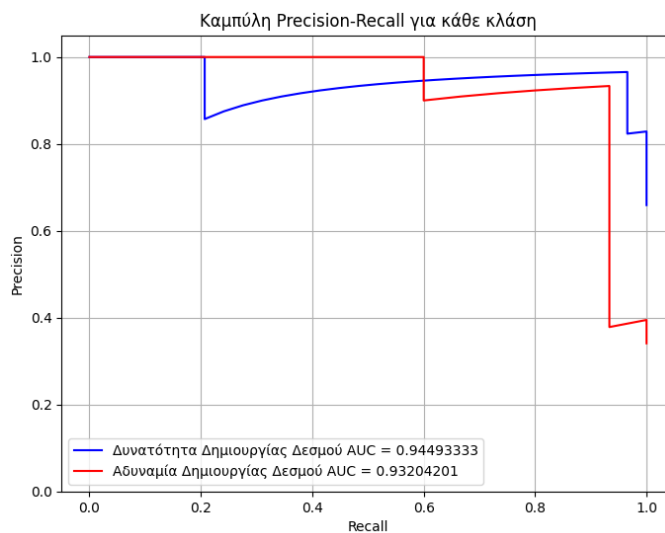
```
Εποχή 1, Απώλεια: 0.4813, Ακρίβεια στο Σετ Επαλήθευσης: 89.52%
Εποχή 2, Απώλεια: 0.3132, Ακρίβεια στο Σετ Επαλήθευσης: 89.52%
Εποχή 3, Απώλεια: 0.2120, Ακρίβεια στο Σετ Επαλήθευσης: 89.52%
Εποχή 4, Απώλεια: 0.1481, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 5, Απώλεια: 0.1043, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 6, Απώλεια: 0.0834, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 7, Απώλεια: 0.0680, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 8, Απώλεια: 0.0625, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 9, Απώλεια: 0.0570, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 10, Απώλεια: 0.0557, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 11, Απώλεια: 0.0637, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 12, Απώλεια: 0.0515, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 13, Απώλεια: 0.0533, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 14, Απώλεια: 0.0526, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 15, Απώλεια: 0.0511, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 16, Απώλεια: 0.0509, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 17, Απώλεια: 0.0505, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 18, Απώλεια: 0.0505, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 19, Απώλεια: 0.0537, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 20, Απώλεια: 0.0533, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
```

```
Test Loss: 0.30640, Test Accuracy: 95.4545%
```



Βλέπουμε ότι το μοντέλο έχει μία αρκετά καλή απόδοση, καθώς φτάνει 90% ακρίβεια. Ταυτόχρονα το μοντέλο έχει πολύ καλή γενίκευση καθώς στο τεστ σετ έχει ακρίβεια 95% κάνοντας μόλις 2 λάθη. Βέβαια πρέπει να λάβουμε υπόψη ότι το τεστ σετ είναι πολύ μικρότερο.

Ακόμη υπολογίστηκε και απεικονίστηκε η καμπύλη Precision-Recall για κάθε κλάση ξεχωριστά:



Εδώ βλέπουμε ότι το μοντέλο έχει μία καλύτερη συμπεριφορά για τα δεδομένα που αντιστοιχούν σε χημικά μόρια που μπορούν να δημιουργήσουν αποτελεσματικά δεσμούς με βιολογικούς υποδοχείς. Για τα δεδομένα που αντιστοιχούν σε χημικά μόρια που δεν μπορούν να δημιουργήσουν δεσμούς έχει επίσης σχετικά καλή συμπεριφορά όσον αφορά της μετρικές precision-recall, αλλά όχι τόσο καλή όσο για την άλλη κλάση.

Σε μία προσπάθεια εύρεσης πιο αποτελεσματικού δικτύου εισήγαγα την βιβλιοθήκη ortuna, η οποία δίνει την δυνατότητα εκτέλεσης Bayesian αναζήτησης, η οποία μελετά συνδυασμούς οι οποίοι έχουν μεγαλύτερη πιθανότητα να έχουν καλά αποτελέσματα. Είναι προτιμότερη αυτή η αναζήτηση έναντι της GridSearch, αφού στην προκειμένη περίπτωση οι συνδυασμοί που είναι απαραίτητο να δοκιμαστούν είναι πάρα πολλοί για να τους δοκιμάσουμε τυχαίο τρόπο.

Η αναζήτηση έγινε για την εύρεση των καλύτερων τιμών για τις παρακάτω παραμέτρους:

- Αριθμός κρυφών επιπέδων (Από 1 έως και 3)
- Αριθμός νευρώνων για κάθε κρυφό επίπεδο (Από 8 έως 512)
- Τιμή πιθανότητας dropout για κάθε κρυφό επίπεδο (Από 0 έως και 0.5)
- Αριθμός εποχών εκπαίδευσης (Από 5 έως και 25)
- Ρυθμός μάθησης (Από 0.1 έως και 0.00001)

Μάλιστα για κάθε συνδυασμό εκπαιδεύτηκαν 5 μοντέλα και κρατήθηκε ο συνδυασμός που είχε τον υψηλότερο μέσο όρο για την ακρίβεια στα δεδομένα επαλήθευσης.

Από την εκτέλεση της αναζήτησης ξεχώρισαν οι παρακάτω συνδυασμοί:

- 1) Trial 19 finished with value: 0.9096774193548386 and parameters: {'n_epochs': 8, 'n_layers': 3, 'n_units_l0': 172, 'dropout_l0': 0.3007434217546421, 'n_units_l1': 221, 'dropout_l1': 0.17261217285850217, 'n_units_l2': 73, 'dropout_l2': 0.2900352997960147, 'lr': 0.00944858945228391}
- 2) Trial 64 finished with value: 0.9096774193548386 and parameters: {'n_epochs': 10, 'n_layers': 3, 'n_units_l0': 124, 'dropout_l0': 0.3126344670015665, 'n_units_l1': 368, 'dropout_l1': 0.2955945936560984, 'n_units_l2': 25, 'dropout_l2': 0.13453130120434686, 'lr': 0.002120891987480904}
- 3) Trial 82 finished with value: 0.9096774193548386 and parameters: {'n_epochs': 9, 'n_layers': 3, 'n_units_l0': 193, 'dropout_l0': 0.31464985079425517, 'n_units_l1': 412, 'dropout_l1': 0.38445044286719954, 'n_units_l2': 72, 'dropout_l2': 0.12359777308955118, 'lr': 0.006180205970557733}

Πριν προχωρήσω στην δημιουργία νέου δικτύου, έκανα μία δεύτερη αναζήτηση πιο περιορισμένη με βάση τα καλύτερα μοντέλα της πρώτης.

Η νέα περιοχή αναζήτησης ορίζεται από τις παρακάτω παραμέτρους:

- Αριθμός κρυφών επιπέδων = **3**
- Αριθμός νευρώνων για το κρυφό επίπεδο 1: **150-200**
- Πιθανότητα Dropout για το κρυφό επίπεδο 1: **0.25-0.35**
- Αριθμός νευρώνων για το κρυφό επίπεδο 2: **200-400**
- Πιθανότητα Dropout για το κρυφό επίπεδο 2: **0.15-0.35**
- Αριθμός νευρώνων για το κρυφό επίπεδο 3: **30-75**
- Πιθανότητα Dropout για το κρυφό επίπεδο 3: **0.15-0.26**
- Αριθμός εποχών εκπαίδευσης = **10**
- Ρυθμός μάθησης: **0.0001-0.005**

Τα αποτελέσματα που ξεχώρισαν σε αυτή την αναζήτηση είναι τα παρακάτω:

- 1) Trial 19 finished with value: 0.9096774193548386 and parameters: {'n_units_l0': 173, 'dropout_l0': 0.3274213551994359, 'n_units_l1': 285, 'dropout_l1': 0.244857458885849, 'n_units_l2': 70, 'dropout_l2': 0.18023449664277866, 'lr': 0.004984853519645431}.
- 2) Trial 20 finished with value: 0.9096774193548386 and parameters: {'n_units_l0': 174, 'dropout_l0': 0.33130584276554, 'n_units_l1': 280, 'dropout_l1': 0.19603780238107624, 'n_units_l2': 70, 'dropout_l2': 0.17737164767382213, 'lr': 0.004872942258657056}.
- 3) Trial 61 finished with value: 0.9096774193548386 and parameters: {'n_units_l0': 176, 'dropout_l0': 0.31685876073851904, 'n_units_l1': 272, 'dropout_l1': 0.24043003220090287, 'n_units_l2': 65, 'dropout_l2': 0.19780132234980302, 'lr': 0.0012909365556269831}.
- 4) Trial 66 finished with value: 0.9096774193548386 and parameters: {'n_units_l0': 200, 'dropout_l0': 0.3186542952978583, 'n_units_l1': 269, 'dropout_l1': 0.2065505153531098, 'n_units_l2': 57, 'dropout_l2': 0.1922309039856285, 'lr': 0.004956860784256152}.
- 5) Trial 81 finished with value: 0.9096774193548386 and parameters: {'n_units_l0': 172, 'dropout_l0': 0.3261838087289948, 'n_units_l1': 269, 'dropout_l1': 0.24038499854513165, 'n_units_l2': 65, 'dropout_l2': 0.2161408168490179, 'lr': 0.0012145908323711462}.
- 6) Trial 86 finished with value: 0.9096774193548386 and parameters: {'n_units_l0': 179, 'dropout_l0': 0.32951050600728266, 'n_units_l1': 276, 'dropout_l1': 0.1836746614416425, 'n_units_l2': 62, 'dropout_l2': 0.2324281758789875, 'lr': 0.0034498839145360623}.

Με βάση αυτά αποφάσισα το τελικό νευρωνικό δίκτυο να έχει την παρακάτω αρχιτεκτονική:

```
class ModifiedNeuralNetwork(nn.Module):
    def __init__(self):
        super(ModifiedNeuralNetwork, self).__init__()
        self.fc1 = nn.Linear(data_train_lda_df.shape[1], 175)
        self.drop1 = nn.Dropout(0.32)
        self.fc2 = nn.Linear(175, 275)
        self.drop2 = nn.Dropout(0.2)
        self.fc3 = nn.Linear(275, 65)
        self.fc4 = nn.Linear(65, 1)

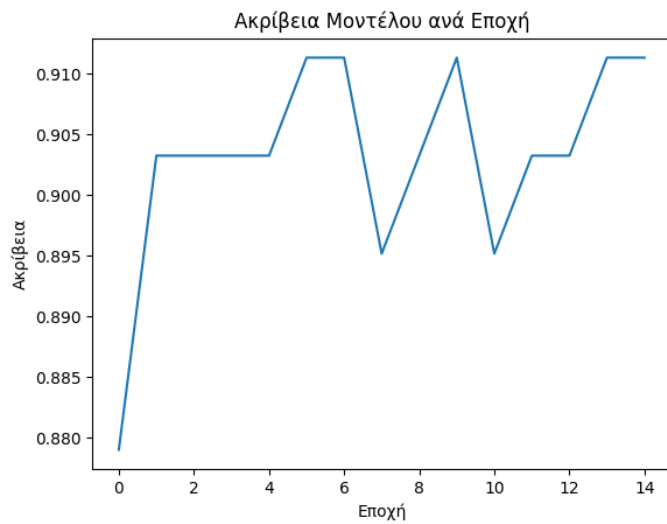
    def forward(self, x):
        x = self.drop1(torch.relu(self.fc1(x)))
        x = self.drop2(torch.relu(self.fc2(x)))
        x = self.drop2(torch.relu(self.fc3(x)))
        x = torch.sigmoid(self.fc4(x))
        return x
```

Το νευρωνικό αυτό έχει ένα ακόμα κρυφό επίπεδο, ενώ οι νευρώνες κάθε επιπέδου αρκετά περισσότεροι από αυτούς του αρχικού μοντέλου. Ακόμα προστέθηκε η λειτουργία dropout στο πρώτο κρυφό επίπεδο με πιθανότητα 32% και στο 2^ο και το 3^ο κρυφό επίπεδο με πιθανότητα 25%.

Ορίζω ένα μοντέλο που να ακολουθά την νέα αρχιτεκτονική νευρωνικού δικτύου, τις παραμέτρους του και εκτελώ την εκπαίδευση και την αξιολόγησή του:

```
model_2 = ModifiedNeuralNetwork()

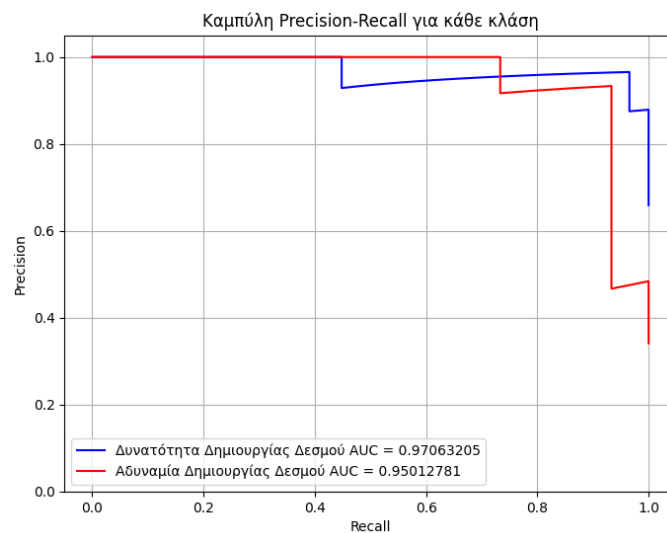
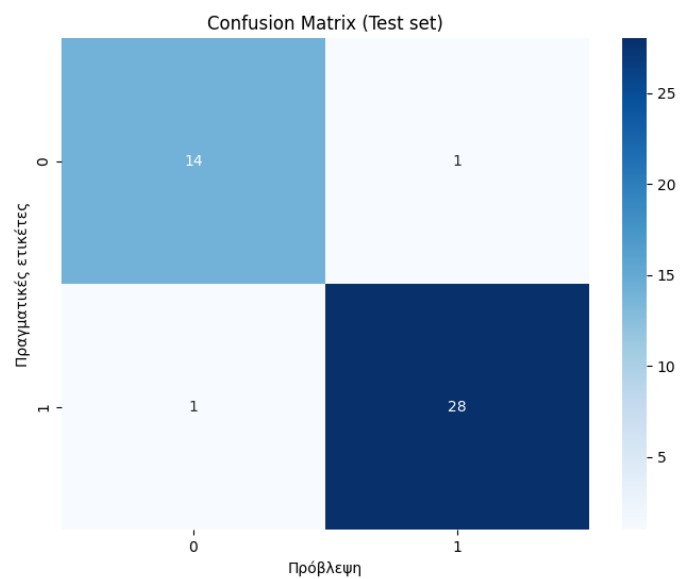
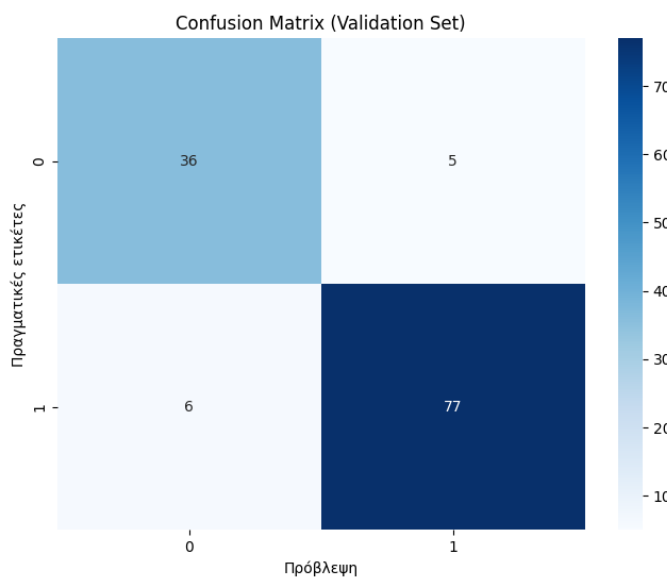
# Συνάρτηση Απώλειας και Βελτιστοποιητής
criterion = nn.BCELoss()
learning_rate = 0.002
optimizer = optim.Adam(model_2.parameters(), lr=learning_rate)
```



Εποχή 1, Απώλεια: 0.2499, Ακρίβεια στο Σετ Επαλήθευσης: 87.90%
Εποχή 2, Απώλεια: 0.0661, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 3, Απώλεια: 0.0775, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 4, Απώλεια: 0.0624, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 5, Απώλεια: 0.0591, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 6, Απώλεια: 0.0537, Ακρίβεια στο Σετ Επαλήθευσης: 91.13%
Εποχή 7, Απώλεια: 0.0544, Ακρίβεια στο Σετ Επαλήθευσης: 91.13%
Εποχή 8, Απώλεια: 0.0620, Ακρίβεια στο Σετ Επαλήθευσης: 89.52%
Εποχή 9, Απώλεια: 0.0554, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 10, Απώλεια: 0.0522, Ακρίβεια στο Σετ Επαλήθευσης: 91.13%
Εποχή 11, Απώλεια: 0.0540, Ακρίβεια στο Σετ Επαλήθευσης: 89.52%
Εποχή 12, Απώλεια: 0.0524, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 13, Απώλεια: 0.0571, Ακρίβεια στο Σετ Επαλήθευσης: 90.32%
Εποχή 14, Απώλεια: 0.0546, Ακρίβεια στο Σετ Επαλήθευσης: 91.13%
Εποχή 15, Απώλεια: 0.0517, Ακρίβεια στο Σετ Επαλήθευσης: 91.13%

Test Loss: 0.37995, Test Accuracy: 95.4545%

Το νέο αυτό μοντέλο έχει ελαφρώς καλύτερη απόδοση κατά μέσο όρο και η ακρίβεια του αγγίζει αρκετά συχνά το 91% (όπως φαίνεται και στην εικόνα). Ακόμη οι πίνακες σύγχυσης για τα δύο σετ δεδομένων (επαλήθευσης και δοκιμής) δίνονται στην συνέχεια μαζί με τις καμπύλες Precision-Recall για τα δεδομένα δοκιμής:



Παρατηρούμε μία βελτίωση στην γενική συμπεριφορά του ταξινομητή, αλλά η απόδοση του στα δεδομένα τα οποία δεν μπορούν να δημιουργήσουν δεσμούς με βιολογικούς υποδοχείς παραμένουν εμφανώς χειρότερα. Αυτό πολύ πιθανώς να συμβαίνει λόγω της ανισορροπίας που υπάρχει μεταξύ των δεδομένων, η οποία δεν μπορούμε να γνωρίζουμε αν είναι αντιπροσωπευτική του πραγματικού προβλήματος, καθώς αυτό έγκειται σε γνώσεις άλλου πεδίου άγνωστου για εμάς. Λόγω αυτής της έλλειψης γνώσεων πάνω στην φύση του προβλήματος, θεωρήθηκε ότι τα δεδομένα είναι αντιπροσωπευτικά, δηλαδή ότι γενικά οι χημικές ενώσεις που μπορούν να δημιουργήσουν αποτελεσματικά δεσμούς με υποδοχείς βρίσκονται σε διπλάσια συχνότητα από ότι οι χημικές ενώσεις που δεν έχουν την ικανότητα αυτή.

SVM

Για μία πρώτη προσπάθεια, δημιουργήθηκε και πάλι ένας SVM, χωρίς να ορίσω καμία παράμετρο. Ήθελα σε πρώτη φάση να δω ένας τυχαίος SVM πόσο καλή απόδοση έχει αν εκπαιδευτεί πάνω στα δεδομένα εκπαίδευσης. Η αξιολόγηση του έγινε εξίσου στο σετ επαλήθευσης και στο σετ δοκιμής.

Το πρώτο μοντέλο είχε την παρακάτω απόδοση στα δύο σετ:

Accuracy on Validation Set: 0.9032				
	precision	recall	f1-score	support
0	0.84	0.88	0.86	41
1	0.94	0.92	0.93	83
accuracy			0.90	124
macro avg	0.89	0.90	0.89	124
weighted avg	0.90	0.90	0.90	124

Accuracy on Test Set: 0.9545				
	precision	recall	f1-score	support
0	0.93	0.93	0.93	15
1	0.97	0.97	0.97	29
accuracy			0.95	44
macro avg	0.95	0.95	0.95	44
weighted avg	0.95	0.95	0.95	44

Στην συνέχεια ορίστηκε ένα GridSearch το οποίο εκτέλεσα και στο τέλος κράτησα τις παραμέτρους που έτρεξαν καλύτερα για τον SVM ταξινομητή και δημιούργησα τον παρακάτω SVM, ο οποίος δεν έχει καλύτερα αποτελέσματα από τον αρχικό, αλλά πλέον έχουμε συγκεκριμένες παραμέτρους.

```
#SVM μοντέλο που βρέθηκε με καλά χαρακτηριστικά
svm_model = SVC(C=0.1, kernel='linear', gamma=0.001, probability=True)
```

Εν συντομία οι παράμετροι έχουν τους παρακάτω λειτουργίες:

Παράμετρος C (Box Constraint): Το box constraint είναι μία σημαντική παράμετρος για την εκπαίδευση του SVM ταξινομητή. Αυτό που κάνει ουσιαστικά είναι να ρυθμίζει τον συμβιβασμό μεταξύ του μεγαλύτερου δυνατού διαχωρισμού των κλάσεων (μεγιστοποίηση την απόσταση και των 2 κλάσεων από το επίπεδο απόφασης) και της ελαχιστοποίησης του αριθμού των δεδομένων που θα παραβιάζουν των κανόνα απόφασης (θα βρίσκονται στην λάθος πλευρά του επιπέδου).

Kernel: Εδώ επιλέγουμε το σχήμα που θα έχει ο πυρήνας, πρακτικά το όριο που θα διαχωρίζει τις δύο κλάσεις. Κάθε πυρήνας λειτουργεί καλύτερα σε διαφορετικούς τύπους δεδομένων. Οπότε έπρεπε να βρούμε αυτόν που ταιριάζει στην φύση των δικών μας δεδομένων.

gamma: Η παράμετρος αυτή παίζει ρόλο κυρίως σε άλλους Kernel και δεν έχει ιδιαίτερη επίδραση στον γραμμικό, οπότε μπορεί να αγνοηθεί:

```
svm_model = SVC(C=0.1, kernel='linear', probability=True)
```


Probability=True: Η παράμετρος αυτή ορίζει ότι θέλουμε να υπολογίζεται η πιθανότητα με την οποία ανήκει ένα στοιχείο στην κλάση που ταξινομείται από τον SVM.

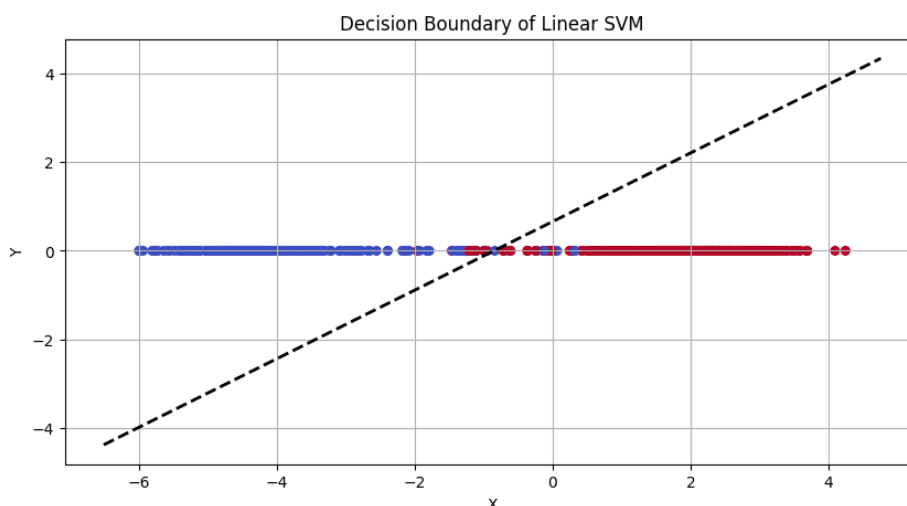
Τα αποτελέσματα αυτού του ταξινομητή, όπως είπα δεν διαφέρουν με τον αρχικό SVM, αλλά δίνονται:

Accuracy on Validation Set: 0.9032				
	precision	recall	f1-score	support
0	0.84	0.88	0.86	41
1	0.94	0.92	0.93	83
accuracy			0.90	124
macro avg	0.89	0.90	0.89	124
weighted avg	0.90	0.90	0.90	124

Accuracy on Test Set: 0.9545				
	precision	recall	f1-score	support
0	0.93	0.93	0.93	15
1	0.97	0.97	0.97	29
accuracy			0.95	44
macro avg	0.95	0.95	0.95	44
weighted avg	0.95	0.95	0.95	44

Κάτι που αξίζει να αναφερθεί είναι ότι και SVM με διαφορετικό πυρήνα φάνηκε να φτάνουν την ίδια ακρίβεια, αλλά προτιμήθηκε αυτός με τον γραμμικό πυρήνα γιατί πέρα από την ίδια απόδοση με τους άλλους, είναι και ο φθηνότερος υπολογιστικά.

Παρακάτω παρατίθεται μία απεικόνιση του SVM για τα δεδομένα εκπαίδευσης:



Συνδυαστική Μέθοδος

Σε αυτό το μέρος δοκιμάστηκε να χρησιμοποιηθούν οι προβλέψεις και των δύο μοντέλων για την βελτίωση των αποτελεσμάτων.

Μετά από την υλοποίηση και από αρκετές εκτελέσεις αποδείχτηκε ότι η ακρίβεια του μοντέλου δεν αυξήθηκε. Αυτό που αυξήθηκε όμως είναι η σταθερότητα του. Ενώ χρησιμοποιώντας τον κάθε ταξινομητή ξεχωριστά ο συνδυασμός των αποτελεσμάτων των δύο μοντέλων, κρατώντας τα αποτελέσματα για το πιο 'confident' μοντέλο κάθε φορά αύξησε την μέση απόδοση των μοντέλων.

Με την έννοια confident εννοώ ότι εισάγοντας ένα δεδομένο στα δύο μοντέλα, το καθένα από αυτά παράγει μία τιμή από το 0 μέχρι το 1, αν η τιμή είναι κάτω από το 0.5 το δεδομένο κατηγοριοποιείται ως 0, ενώ αν η τιμή είναι μεγαλύτερη του 0.5, το δεδομένο κατηγοριοποιείται ως 1.

Πλέον αφού πάρω την έξοδο κάθε μοντέλου συγκρίνω πιο είναι μακριά από την τιμή 0.5, δηλαδή πιο εφαρμόζει την ταξινόμηση με μεγαλύτερη σιγουριά και επιλέγω την γνώμη εκείνου του μοντέλου. Τα αποτελέσματα της μεθόδου δίνονται εδώ:

Accuracy on Validation Set: 0.9032
Accuracy on Test Set: 0.9545

Πριν προχωρήσω στην επιλογή του μοντέλου έλεγξα πρώτα να δω τις διαφορές για τις προβλέψεις των μοντέλων στο σετ επαλήθευσης:

```
differences = compare_lists(np.array(all_predictions), predictions_validation, np.array(most_confident_predictions_val))
print(f'Διαφορές μεταξύ λίστας 1 και λίστας 2: {differences[0]}')
print(f'Διαφορές μεταξύ λίστας 1 και λίστας 3: {differences[1]}')
print(f'Διαφορές μεταξύ λίστας 2 και λίστας 3: {differences[2]}')
```

```
Διαφορές μεταξύ λίστας 1 και λίστας 2: 1
Διαφορές μεταξύ λίστας 1 και λίστας 3: 1
Διαφορές μεταξύ λίστας 2 και λίστας 3: 2
```

Όπως φαίνεται:

1. Οι προβλέψεις του νευρωνικού μοντέλου (all_prediction) με τις προβλέψεις του SVM (predictions_validation) διαφέρουν κατά μία πρόβλεψη.
2. Οι προβλέψεις του νευρωνικού μοντέλου με τις προβλέψεις του συνδυασμού των μοντέλων (most_confident_predictions_val) διαφέρουν κατά 1 στοιχείο.
3. Οι προβλέψεις του SVM ταξινομητή με τις προβλέψεις του συνδυασμού των μοντέλων διαφέρουν κατά 2 στοιχεία.

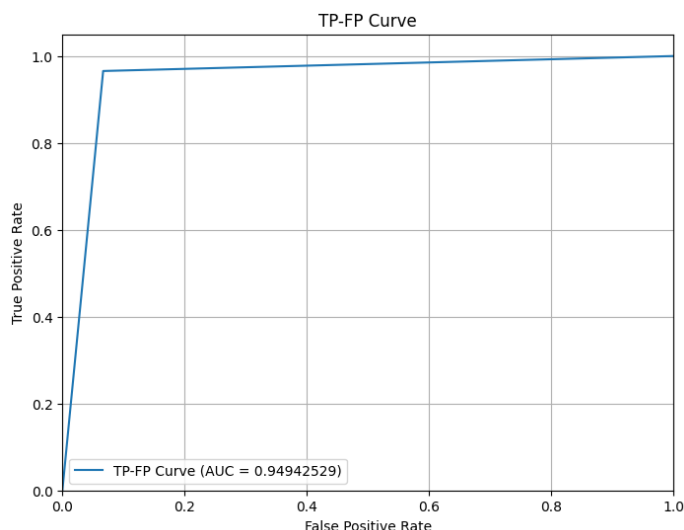
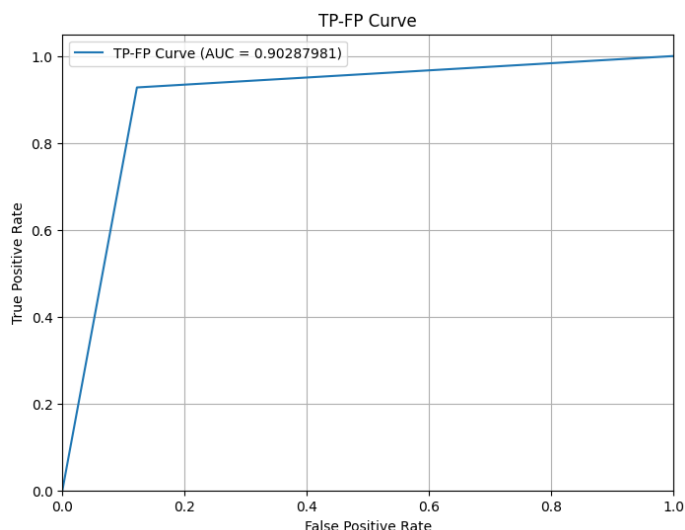
Ομοίως γίνεται και για το σετ δοκιμής:

```
differences = compare_lists(np.array(all_predictions_test), predictions_test, np.array(most_confident_predictions_test))
print(f'Διαφορές μεταξύ λίστας 1 και λίστας 2: {differences[0]}')
print(f'Διαφορές μεταξύ λίστας 1 και λίστας 3: {differences[1]}')
print(f'Διαφορές μεταξύ λίστας 2 και λίστας 3: {differences[2]}')
```

```
Διαφορές μεταξύ λίστας 1 και λίστας 2: 1
Διαφορές μεταξύ λίστας 1 και λίστας 3: 0
Διαφορές μεταξύ λίστας 2 και λίστας 3: 1
```

Πριν κλείσουμε την εργασία πρέπει να γίνει και μία αξιολόγηση. Για κάθε μοντέλο υπολογίστηκε η καμπύλη TP-FP για τα σετ επαλήθευσης και δοκιμής.

Νευρωνικό Δίκτυο

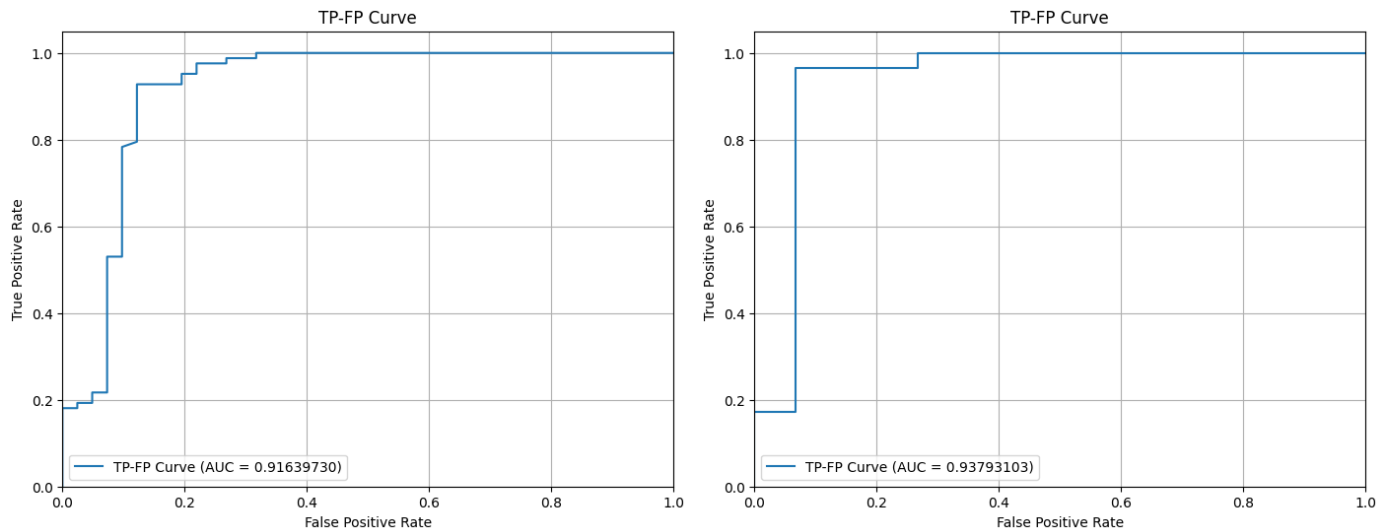


(Αριστερά σετ επαλήθευσης και δεξιά σετ δοκιμής)

Παρατηρούμε ότι σε γενικές γραμμές το νευρωνικό έχει καλή συμπεριφορά για τις πραγματικές ετικέτες 1, αφού το επιθυμητό σε αυτή τη καμπύλη είναι να είναι από πολύ νωρίς (όσο πιο αριστερά γίνεται) κοντά στην τιμή 1. Αυτό φαίνεται να γίνεται.

Η τιμή AUC είναι επίσης αρκετά υψηλή δείχνοντας μας ότι το μοντέλο λειτουργεί αρκετά καλά.

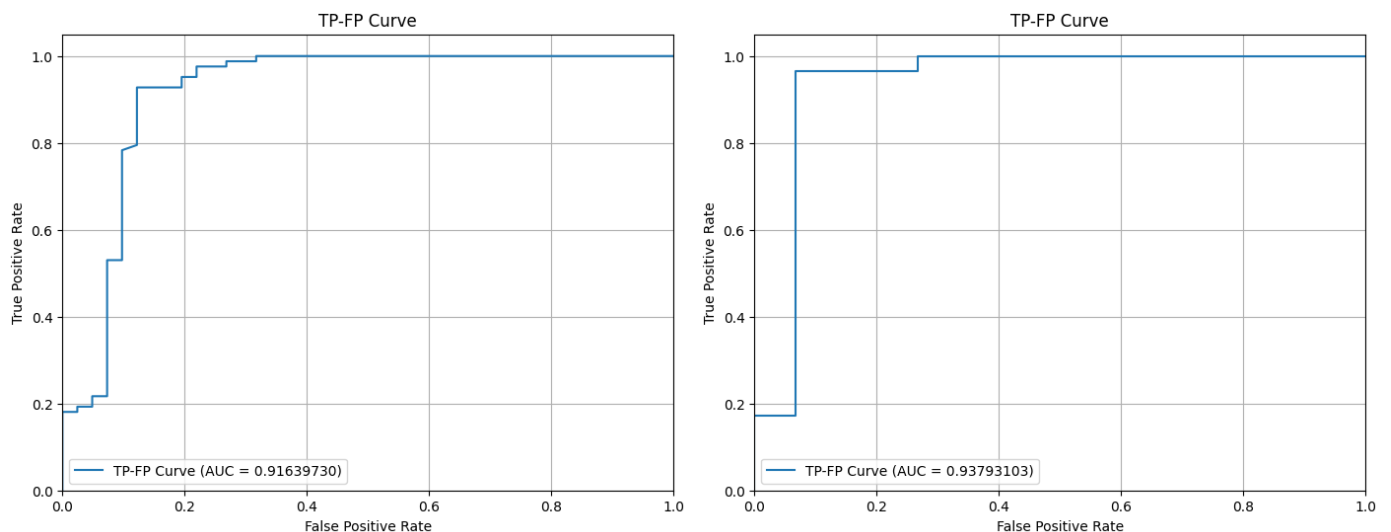
SVM Ταξινομητής



(Αριστερά σετ επαλήθευσης και δεξιά σετ δοκιμής)

Εδώ τα αποτελέσματα φαίνονται αρκετά καλύτερα από αυτά του νευρωνικού δικτύου.

Συνδυαστικό μοντέλο



Τα αποτελέσματα είναι τα ίδια με αυτά του με αυτά του SVM, καθώς φαίνεται να έχει εξάγει πιο σίγουρες προβλέψεις.

Τα αποτελέσματα για την πρόβλεψη τα κράτησα από την συνδυαστική μέθοδο, καθώς οι αποφάσεις που παίρνονται είναι πιο σίγουρες για τα ίδια τα μοντέλα.

Αναμένεται μία απόδοση ανάμεσα στο 91% και 93%. Αυτό γιατί παρόλο που στο τεστ σετ παρατηρείται απόδοση έως και 95.5%, πρέπει να ληφθεί υπόψη ότι το test set είναι πολύ μικρότερο από το validation set.